

테스트 - 타임 메모리 계열 — Titans 에서 Sleep 까지 , 그리고 서빙 비용

학습을 처음 보는 inference·systems 엔지니어를 위한 6 편 (Titans·Miras·Atlas·TNT·Nested Learning·Sleep) 해부
+ 배포 비용 모델링

이승호 (SAIT) · 2 시간 발표 + 30 분 Q&A

오늘 다룰 것 — 세 파트

A. Background

학습이 처음이라면

A1. 학습 한 바퀴 (2-layer MLP)

·
forward→loss→backward→optimizer→update

A2. Optimizer 의 역사 (수식)

- 계보 지도 → GD·SGD·Momentum
- Adagrad·RMSProp·Adam·AdamW

A3. 대안 계열 (개념별 1 장)

- linear attention · DeltaNet · Gated DeltaNet
- SSM · Mamba · Mamba-2

B. 논문 6 편

핵심 발명 → 논거 → 증명

B1. Titans

- test-time 신경망 메모리 · MAC/MAG/MAL

B2. Miras

- 4 축 설계공간 (메모리=online 최적화)

B3. Atlas

- Omega rule · 용량 확장 · Muon

B4. TNT

- chunk 정렬 · global/local · serving

B5. Nested Learning / HOPE

- optimizer=memory · self-mod · CMS

B6. Sleep

- offline consolidation · Dreaming

C. System modeling

우리 기여 — 서빙 비용

C0. 방법론 (HATIR · HAT schema)

- 무엇으로 뽑았고 왜 신뢰할 수 있나

C1. pair thesis — decode=memory-bound

C2. 8 개 가속기 크로스 측정

C3. HOPE-block roofline (H100 트윈)

C4. zHBM · HBM-PIM · SRAM scaling

C5. scaling 청사진

이 세미나의 목적 · 범위 · 주의

목적 · 방향

inference·systems 엔지니어가 test-time memory 계열 (Titans~Sleep) 을 shape· 비용 수준으로 이해한다 . 각 논문의 핵심 발명을 먼저 보고
→ 어떻게 구현 · 증명되는지 → 논문 간 framing 이 이어지는 흐름 → 마지막에 이 배포 형태가 서빙에서 얼마나 드는지 (우리 기여).

미리 알아둘 3 가지 (읽는 규칙)

① 학습 (pre-training) 은 다루지 않는다

Titans 기반 모델을 처음부터 훈련하는 과정은 범위 밖 . 이 계열의 test-time 동작과 서빙에 집중한다 . (Background 의 학습 설명은 ' 무엇을 배우는가 ' 를 읽기 위한 최소한 .)

② 논문에 구체 수치가 빠진 곳이 많다

모델 파라미터 shape· 차원 · 학습 하이퍼 등 구체값이 명시 안 된 부분이 흔하다 . 그림 · 설명에서 추론한 것은 ' 추론 ' 으로 , 본문에 있는 것은 ' 명시 ' 로 구분해 표시한다 .

③ 논문 1 편만으로 불완전하다

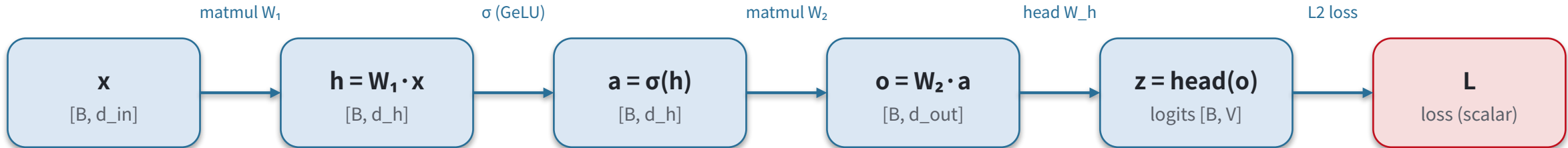
한 논문의 설명이 스스로 완결되지 않아 , 다른 논문 (특히 HOPE) 을 같이 봐야 이해되는 부분이 많다 . 예 : Titans 의 투영 분리는 HOPE 에서야 명시된다 .
그래서 6 편을 하나의 흐름으로 읽는다 .

PART A

Background — 학습이 처음이라면

2-layer MLP + head: forward → loss → backward → optimizer → update

① forward



batch B 개 샘플 동시 처리 · loss 는 B 개 평균 (또는 합) 으로 누적

② loss.backward() — autograd 역전파

출력 쪽 $\partial L / \partial z$ 부터 체인 룰로 거슬러 $\partial L / \partial W_2 \rightarrow \partial L / \partial a \rightarrow \partial L / \partial h \rightarrow \partial L / \partial W_1$ 까지. 각 파라미터 W 의 gradient $g = \partial L / \partial W$ 를 구해 W.grad 에 채운다 (순전파의 역순).

③ optimizer.step() — AdamW (g 를 받아 '얼마나 · 어느 방향' 결정)

$$m \leftarrow \beta_1 m + (1 - \beta_1) g$$

← 1 차 모멘텀 = 방향

$$v \leftarrow \beta_2 v + (1 - \beta_2) g^2$$

← 2 차 모멘텀 = 보폭

$$\Delta W = -\eta \frac{\hat{m}}{\sqrt{\hat{v}} + \epsilon} - \eta \lambda W$$

← decoupled weight decay

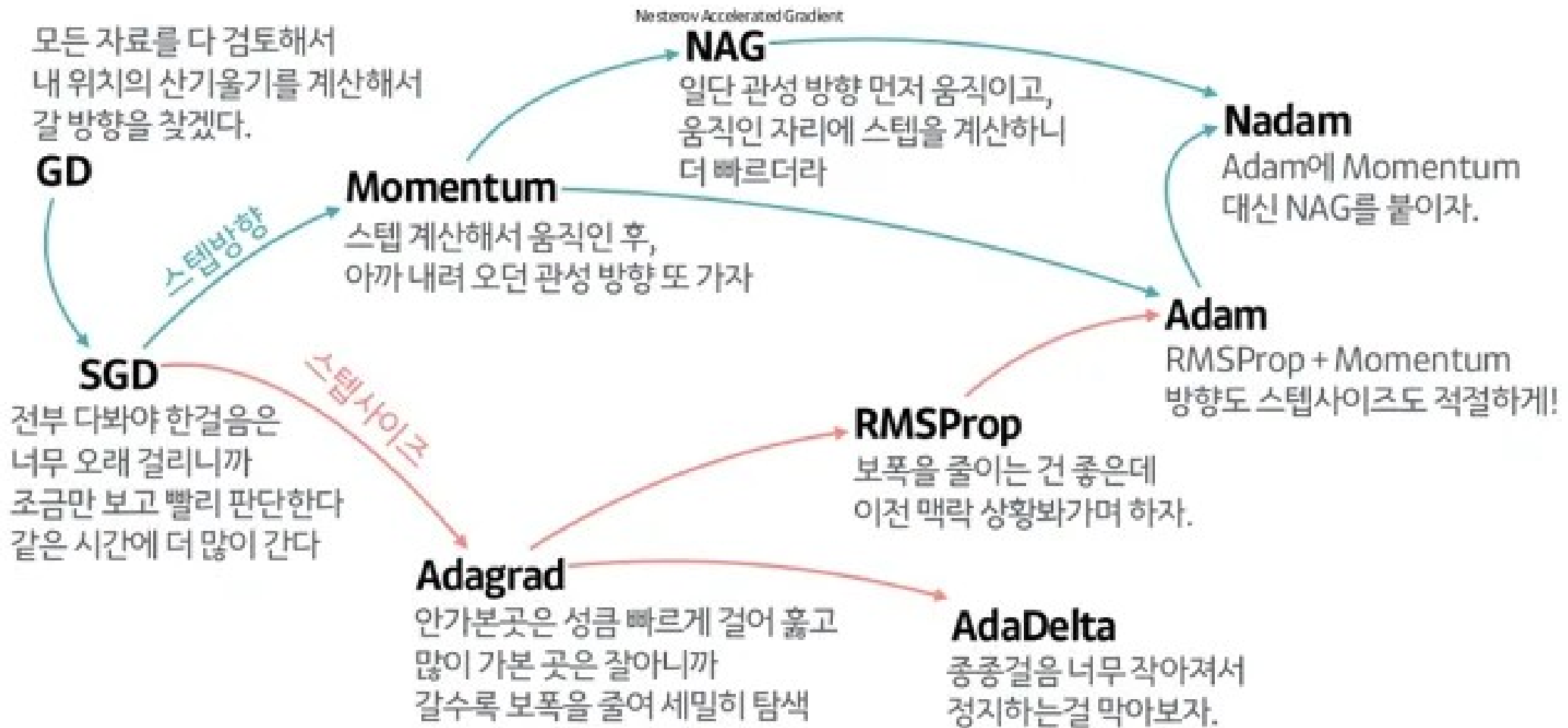
④ $W \leftarrow W + \Delta W$

W_1, W_2, head 를 갱신.
optimizer.zero_grad() 로 g 비우고
다음 batch 로 → 반복.

핵심 한 문장

학습 = ①forward 예측 → ② backward 로 gradient(어디가 틀렸나) → ③optimizer 로 가공 (m,v) 해 갱신량 → ④ weight 이동. 이 계열은 ②③④를 '추론 중에도' 돌린다.

한 장으로 보는 직관 지도



파란 줄기 = '어느 방향으로' (관성) 를 고치는 흐름 · 초록 줄기 = '얼마나 (보폭)' 를 고치는 흐름 → Adam 이 둘을 합침 . 다음 장부터 이 중 GD·SGD·Momentum·Adagrad·RMSProp·Adam·AdamW 를 1 페이지씩 수식과 함께 .

GD · Gradient Descent (batch)

뿌리 — 가장 정확한 한 걸음

직관 (왜 이렇게 ?)

" 모든 자료를 다 검토해서 , 내 위치의 산 기울기를 계산해 갈 방향을 찾았다 ."

갱신 규칙 (수식)

$$g_t = \frac{1}{N} \sum_{i=1}^N \nabla \ell_i(\theta_t) \quad \text{전체 } N \text{ 개 데이터의 평균 gradient}$$

$$\theta_{t+1} = \theta_t - \eta g_t \quad \eta = \text{learning rate (보폭)}$$

이 optimizer 가 해결한 것

손실을 가장 빨리 줄이는 방향 (가장 가파른 내리막) 을 데이터 전체로 정확히 계산한다 .

남은 문제 → 다음

한 스텝마다 데이터 N 개를 전부 봐야 함 → 너무 느리다 . → SGD

SGD · Stochastic Gradient Descent

파란 줄기 — 속도

직관 (왜 이렇게 ?)

" 전부 다 봐야 한 걸음은 너무 오래 걸리니까 , 조금만 보고 빨리 판단한다 . 같은 시간에 더 많이 간다 ."

갱신 규칙 (수식)

$$g_t = \nabla \ell_B(\theta_t) \quad \text{미니배치 } B \text{ 개만 사용 } (|B| \ll N)$$

$$\theta_{t+1} = \theta_t - \eta g_t$$

이 optimizer 가 해결한 것

스텝당 비용이 급감 → 같은 시간에 훨씬 많은 스텝 . gradient noise 가 얇은 지역최소 탈출에도 도움 .

남은 문제 → 다음

gradient 가 매 스텝 출렁여 지그재그 . 방향도 보폭도 여전히 그대로 .
→ Momentum · Adagrad

Momentum · SGD with Momentum

파란 줄기 — '스텝 방향' 개선

직관 (왜 이렇게 ?)

" 스텝을 계산해서 움직인 후 , 아까 내려오던 관성 방향으로 또 가자 ."

갱신 규칙 (수식)

$$v_t = \mu v_{t-1} + g_t \quad \mu \approx 0.9, \text{ 과거 방향의 관성 (누적)}$$

$$\theta_{t+1} = \theta_t - \eta v_t$$

이 optimizer 가 해결한 것

일관된 방향은 누적 가속하고 출렁이는 성분은 상쇄 → 지그재그 완화 , 좁은 골짜기를 빠르게 통과 .

남은 문제 → 다음

보폭 (스케일) 은 아직 모든 좌표가 동일 . ' 좌표별 보폭 ' 은 아래 초록 줄기 (Adagrad→RMSProp) 가 맡고 , 뒤에서 Adam 이 방향과 합친다 .

Adagrad · Adaptive Gradient

초록 줄기 — '스텝 사이즈' 개선

직관 (왜 이렇게 ?)

" 안 가 본 곳은 성큼 빠르게 훑고 , 많이 가 본 곳은 잘 가니까 갈수록 보폭을 줄여 세밀히 탐색 ."

갱신 규칙 (수식)

$$G_t = G_{t-1} + g_t^2 \quad \text{좌표별 gradient}^2 \text{ 누적}$$

$$\theta_{t+1} = \theta_t - \eta \frac{g_t}{\sqrt{G_t} + \epsilon} \quad \text{좌표마다 다른 보폭}$$

이 optimizer 가 해결한 것

좌표마다 다른 learning rate — 드물게 큰 gradient(희소 feature) 는 크게 , 자주 큰 좌표는 작게 .

남은 문제 → 다음

G_t 가 단조 증가 → 분모가 계속 커져 보폭이 0 으로 죽는다 (학습 정지). → RMSProp

RMSProp · Root Mean Square Propagation

초록 줄기 — EMA 로 보완

직관 (왜 이렇게 ?)

" 보폭을 줄이는 건 좋은데 , (전부 누적하지 말고) 최근 맥락을 봐 가며 하자 ."

갱신 규칙 (수식)

$$\mathbb{E}[g^2]_t = \rho \mathbb{E}[g^2]_{t-1} + (1 - \rho) g_t^2 \quad \rho \approx 0.9, \text{ 최근값 지수이동평균 (EMA)}$$

$$\theta_{t+1} = \theta_t - \eta \frac{g_t}{\sqrt{\mathbb{E}[g^2]_t + \epsilon}} \quad \text{분모가 무한정 커지지 않음}$$

이 optimizer 가 해결한 것

합 대신 EMA → 분모가 무한정 커지지 않아 보폭이 죽지 않는다 .
비정상 (non-stationary) 목표에도 안정 .

남은 문제 → 다음

방향 (momentum) 은 아직 안 씀 . 초기 (0 에서 시작) 편향 보정도 없다 . → Adam

Adam · Adaptive Moment Estimation

두 줄기의 합류 ★

직관 (왜 이렇게 ?)

"RMSProp(보폭) + Momentum(방향) 을 합치자 — 방향도 스텝 사이즈도 적절하게 !"

갱신 규칙 (수식)

$$g_t = \nabla L(\theta_t) + \lambda \theta_t \quad \text{weight decay = L2 정규화를 gradient 에 더함}$$

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad \text{방향 } m \cdot \text{보폭 } v$$

$$\hat{m} = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v} = \frac{v_t}{1 - \beta_2^t} \quad \text{초기 0- 편향 보정}$$

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}}{\sqrt{\hat{v}} + \epsilon} \quad \lambda \theta \text{ 가 } \hat{m} \text{ 안에 섞여 } 1/\sqrt{\hat{v}} \text{ 로 스케일됨}$$

이 optimizer 가 해결한 것

방향 (m) + 좌표별 보폭 (v) + 편향 보정을 한 번에 . 튜닝 거의 없이 잘 동작 → 사실상 표준 . weight decay 는 보통 L2 로 적용 (위 g_t).

남은 문제 → 다음

그런데 $\lambda \theta$ 가 $\sqrt{\hat{v}}$ 로 나뉘어 좌표마다 다르게 적용 = 왜곡 . 이 항을 밖으로 빼면 ? → AdamW

AdamW ★ · Adam with decoupled Weight decay

종착점 — 오늘의 표준

직관 (왜 이렇게 ?)

"weight decay(정규화) 를 gradient 에 섞지 말고 , weight 에서 ' 직접 ' 빼자 — 그래야 좌표마다 똑같이 적용된다 ."

갱신 규칙 (수식)

$$(\text{Adam+L2}) \quad \theta_{t+1} = \theta_t - \eta \frac{\hat{m}}{\sqrt{\hat{v}} + \epsilon} \quad g = \nabla L + \lambda \theta \rightarrow \lambda \theta \text{ 가 } \sqrt{\hat{v}} \text{ 에 얽힘 (왜곡)}$$

$$(\text{AdamW}) \quad \theta_{t+1} = \theta_t - \eta \frac{\hat{m}}{\sqrt{\hat{v}} + \epsilon} - \eta \lambda \theta_t \quad -\eta \lambda \theta \text{ 를 분리 } \rightarrow \sqrt{\hat{v}} \text{ 와 무관하게 일정}$$

이 optimizer 가 해결한 것

위 두 식의 차이 = 맨 끝 $-\eta \lambda \theta$ 항 . AdamW 는 decay 를 adaptive 스케일 $(1/\sqrt{\hat{v}})$ 밖으로 빼 정규화가 좌표마다 일관 $\rightarrow$ 일반화 $\uparrow$. 오늘 LLM 표준 .

남은 문제 $\rightarrow$ 다음

여기까지가 ' 표준 optimizer ' . 이 계열 논문은 이 optimizer 자체를 memory 로 재해석한다 (HOPE). $\rightarrow$ Part B

왜 softmax attention 을 벗어나나 — sequence 를 ' 상태 ' 로 압축하는 계열

softmax attention

- 문맥이 길수록 KV 캐시가 선형으로 증가 (메모리 폭발)
- 토큰마다 전체를 다시 봄 → 계산 $O(n^2)$
- 무엇을 ' 기억 ' 할지 압축이 없다 — 전부 보관



고정 크기 ' 상태 ' S 로 압축하는 RNN 류

토큰마다 상태 S 를 ' 어떤 규칙으로 ' 갱신하느냐가 계열을 가른다 . 이 관점이 곧 Titans 의 **memory update(=test-time 학습)** 로 이어진다 .

Linear Attention

S 를 더하기만 (additive)
누적 간섭이 문제

DeltaNet → Gated

delta rule 로 덮어쓰기 · 수정
+ forget gate

SSM · Mamba-2

구조화된 선형 recurrence
선택적 상태 갱신

공통 렌즈 — 다음 6 개를 관통하는 한 문장

모두 " 상태 S 를 토큰마다 갱신하는 선형 recurrence " 다 . 규칙의 차이일 뿐 — 더하기 (linear), 덮어쓰기 (delta), 잊기 (gated), 구조화 (SSM). → Titans 는 이 갱신을 ' 작은 신경망 메모리를 gradient 로 학습 ' 하는 것으로 승격시킨다 (Part B).

* delta rule: 현재 상태가 key 로 조회했을 때 내놓는 값과 목표 value 의 ' 예측오차 ' 만큼만 상태를 고치는 규칙 . 손실 $\frac{1}{2} \| S \cdot k - v \|^2$ 에 대한 gradient step 한 번과 같다 .

Linear Attention · 선형 어텐션 (Katharopoulos 2020)

이게 푸는 문제

softmax attention 은 계산 $O(n^2)$ 이고 KV 캐시가 문맥 길이에 비례해 커진다 . 이를 고정 크기 상태의 $O(n)$ 순환식으로 바꾸고 싶다 .

갱신식

$$S_t = S_{t-1} + \phi(k_t) v_t^\top, \quad o_t = \phi(q_t)^\top S_t$$

기호의 의미

S_t : 상태 행렬 ($d \times d$) = fast weights

$\phi(\cdot)$: 양수 feature map (커널)

k_t, v_t : key·value 벡터

q_t : query, o_t : 출력

물리적 의미

softmax 의 ' 유사도 합 ' 을 외적 $\phi(k)v^\top$ 의 누적으로 대체한다 . 상태 S 에 (key→value) 연상을 계속 더해 , 고정 크기 상태로 무한 문맥을 요약 .

이득

시간 $O(n)$ · 메모리 $O(1)$ (문맥 무관). 순환 형태라 디코드가 토큰당 상수 비용 .

한계 → 뒤에서 HOPE 가 해결

그냥 ' 더하기 ' 만 한다 → 오래된 · 충돌하는 연상이 누적 간섭 (crosstalk) 하고 용량이 포화 . (덮어쓰기가 없음)

DeltaNet · 델타넷 (Schlag/Yang, delta rule)

이게 푸는 문제

linear attention 의 ' 무한 누적 간섭 ' 을 줄이자 . 이미 저장된 연상을 덮어쓰거나 오차만큼 고치자 .

갱신식

$$S_t = S_{t-1} - \beta_t (S_{t-1} k_t - v_t) k_t^\top$$

기호의 의미

$S_{t-1} k_t$: 지금 상태가 k_t 에 내놓는 값

$(S_{t-1} k_t - v_t)$: 예측오차

β_t : 쓰기 강도 (0~1) = learning rate

$k_t k_t^\top$: 그 key 방향에만 갱신

물리적 의미

key k_t 로 조회한 값을 목표 v_t 에 가깝게 ' 오차만큼 ' 수정 . 이는 손실 $\frac{1}{2} \| S \cdot k_t - v_t \|^2$ 에 대한 gradient step 한 번 (β = 학습률) 과 정확히 같다 .

이득

기존 연상을 덮어써 간섭 ↓ → 같은 용량으로 더 정확한 회상 .

한계 → 뒤에서 HOPE 가 해결

오래된 기억을 ' 잊는 ' 장치가 없어 용량이 결국 포화한다 . (forget 이 없음)

Gated DeltaNet · 게이트 델타넷 (+forget)

이게 푸는 문제

DeltaNet 엔 망각이 없어 용량이 포화한다 . 오래된 기억을 서서히 감쇠 (잊기) 하자 .

갱신식

$$S_t = \alpha_t S_{t-1} - \beta_t (S_{t-1} k_t - v_t) k_t^\top$$

기호의 의미

α_t : forget gate (0~1, 데이터 의존)

$\alpha_t \cdot S_{t-1}$: 상태 전체를 감쇠

β_t, k_t : DeltaNet 과 동일

($S k - v$) : 예측오차

물리적 의미

매 스텝 상태를 α_t 배로 감쇠 (오래된 기억 소거) 한 뒤 delta rule 로 새 정보를 기입 . = decay(정규화) 가 붙은 online gradient step.

이득

유한 용량을 재할용 → 긴 문맥에서도 안정 , 최신 정보 반영 .

한계 → 뒤에서 HOPE 가 해결

갱신 규칙 (무엇을 · 얼마나 · 잊을지) 이 여전히 ' 고정된 손실 · 고정 학습률 ' .
규칙 자체를 학습하지는 못한다 .

SSM (S4) · 상태공간 모델 (Structured State Space)

이게 푸는 문제

RNN 은 순차적 · 불안정하고 attention 은 $O(n^2)$. 긴 의존성을 '구조화된 선형 recurrence' 로 안정적 · 병렬적으로 다루자.

갱신식

$$h_t = A h_{t-1} + B x_t, \quad y_t = C h_t$$

기호의 의미

h_t : 은닉 상태 (N 차원)

A: 상태전이 (HiPPO 등으로 구조화)

B: 입력→상태, C: 상태→출력

x_t 입력, y_t 출력

물리적 의미

입력을 구조화된 선형 시스템의 상태에 누적. A 의 구조 (HiPPO) 가 과거를 잘 요약하도록 설계돼, convolution 으로 병렬 학습 · recurrence 로 $O(1)$ 디코드.

이득

긴 의존성을 안정적으로 포착, 선형 시간, 병렬 학습 가능.

한계 → 뒤에서 HOPE 가 해결

A,B,C 가 입력과 무관 (시간 불변) → 내용에 따라 '무엇을 기억할지' 선택하지 못한다.

Mamba (S6) · 선택적 SSM (Selective State Space)

이게 푸는 문제

SSM 의 A,B,C 가 고정이라 내용 선택이 안 된다 . 상태 갱신을 입력에 따라 '선택적' 으로 만들자 .

갱신식

$$(B_t, C_t, \Delta_t) = f(x_t), \quad \bar{A}_t = \exp(\Delta_t A)$$

기호의 의미

Δ_t : 입력 의존 시간간격 = 게이트

B_t, C_t : 입력 의존 투영

$\bar{A}_t$: 이산화된 상태전이

$f(\cdot)$: 작은 투영망

물리적 의미

토큰마다 '얼마나 상태를 갱신 · 유지할지' 를 입력이 정한다 (selective). 중요한 토큰은 크게 반영 , 무의미하면 건너뛴 . hardware-aware 스캔으로 병렬 .

이득

내용 기반 선택 → Transformer 급 품질 + 선형 시간 , 초장문 처리 .

한계 → 뒤에서 HOPE 가 해결

여전히 상태 갱신이 '고정된 선형식 + 선택 게이트' ! 메모리를 gradient 로 '학습' 하지는 않는다 .

Mamba-2 (SSD) · State-Space Duality

이게 푸는 문제

attention 과 SSM 이 따로 논다 . 둘을 이론으로 잇고 (duality), 텐서코어로 더 빠르게 만들자 .

갱신식

$$Y = (L \circ (CB^T)) X \iff \text{linear SSM recurrence}$$

기호의 의미

L : 하삼각 마스크 (decay 구조)

CB^T : attention 유사 행렬

$\circ$: 원소곱 (Hadamard)

X 입력, Y 출력

물리적 의미

SSM 의 순환식이 '마스킹된 attention' 과 수학적으로 등가 (duality) 임을 보인다 → 훨씬 큰 상태를 텐서코어 친화적으로 빠르게 구현 .

이득

attention 의 표현력 + SSM 의 선형성 . 큰 상태를 고속 처리 .

한계 → 뒤에서 HOPE 가 해결

여전히 '고정 규칙' 의 선형 recurrence. "sequence layer = 상태 갱신 규칙"
관점만 확립 — 규칙을 학습으로 승격하는 건 Titans~HOPE 의 몫 .

PART B

논문 6 편 — 핵심 발명 먼저 , 그다음 어떻게 증명했나

Titans — test-time 에 학습되는 신경망 메모리

핵심 발명 / 주장

1

문맥을 KV 에 쌓지 말고 , test-time 에 ' 학습되는 ' deep neural memory(작은 MLP) 에 압축한다
attention 의 $O(n^2) \cdot KV$ 폭발 대신 , 고정 크기 신경망이 무엇을 기억할지 스스로 학습

2

무엇을 · 얼마나 기억할지 = surprise(gradient) + momentum(과거 surprise) + forgetting(decay)
예측이 놀라운 (=gradient 큰) 토큰을 더 기억 , 관성으로 이어가고 , 오래된 건 잊는다

3

메모리를 아키텍처에 꽂는 3 가지 방식 : MAC · MAG · MAL
persistent(작업기억) + core(attention) + contextual(장기 메모리) 의 결합 방식 차이

논거 흐름

linear/KV 의 용량 · 간섭 한계 → 메모리를 ' 학습되는 MLP' 로 → 무엇을 배우나 = $k \rightarrow v$ 연상 ($\| M(k) - v \|^2$) → 얼마나 = surprise + momentum + forget → 병렬화
= momentum 을 associative scan 으로 → 어디에 꽂나 = MAC/MAG/MAL

증명 (뒤에서 논문 그림 · 표로)

언어모델링 perplexity · NIAH(needle) · BABILong(초장문 추론) 에서 baseline 상회 , 메모리 깊을수록 개선

효율이 절실한 바로 그 자리 (긴 문맥) 에서 품질이 무너진다

attention (정확하지만 비쌈)

토큰마다 과거 전체를 다시 본다 → 계산 $O(n^2)$, KV 캐시가 길이에 비례해 증가 . 무한정 길게는 불가능 .

linear recurrent (싸지만 압축)

Mamba · DeltaNet: 길이에 비례하는 싼 비용 . 그러나 과거 전체를 ' 고정 크기 state 하나 ' 에 눌러 담는다 .

근본 모순

싼 비용이 가장 절실한 구간 = 아주 긴 문맥 . 그런데 긴 문맥일수록 작은 state 에 제대로 안 담긴다 → 효율이 필요한 자리에서 품질이 무너지도록 설계돼 있다 .

직전 시도 TTT 의 3 가지 결핍

state 를 작은 모델 weights 로 보고 gradient 로 갱신 — LM 스케일에서 처음 작동 .
그러나 :

- ① 지우기가 없다 (순수 SGD → write 만 무한 누적 → 포화)
- ② optimizer 가 기억을 못 한다 (토큰마다 자기 gradient 하나만 반영)
- ③ deep memory 의 이득을 실험으로 답하지 않았다

다른 한쪽 — DeltaNet 계열

memory 를 matrix 로 묶어 ' 정확한 병렬 학습 공식 ' 을 얻음 . Gated DeltaNet 은 forget gate 까지 추가 .

그러나 딜레마 : ' 표현력 있는 갱신 규칙 + 깊은 비선형 memory ' 와 ' 병렬 학습 가능성 ' 이 양립 불가처럼 보였다 .

→ 그 가운데가 비어 있었다 .

Titans 의 자리

표현력 (깊은 비선형 memory + 똑똑한 갱신 규칙) 과 병렬 학습 가능성을 동시에 — 그 ' 빈 가운데 ' 를 채운다 .

memory = 작은 신경망의 weights, 그 weights 가 recurrent state

관점 : 모든 sequence 모델을 세 요소로 분해

memory 구조 · write(갱신) 연산 · read(검색) 연산 . 질문 : test-time 에 memory 를 gradient 로 갱신하고 그 규칙을 잘 설계하면 고정 state 의 한계를 넘을 수 있지 않을까 ?

memory 가 푸는 문제 = associative memory regression

작은 MLP(long-term memory module, LMM) 의 weights = state.

토큰을 key/value 로 투영하고 , memory 가 key 를 넣으면 value 를 재생하도록 학습 :

$$\ell(M_{t-1}; x_t) = \|M_{t-1}(k_t) - v_t\|^2$$

결정적 구분 — 두 층위

key/value/query 투영 weights = inner 문제의 hyperparameter.

→ inference 중엔 절대 안 움직이고 pre-training 값으로 고정 .

실제로 매 토큰 움직이는 건 memory 의 weights 뿐이다 .

무엇을 얼마나 세계 외일까 = surprise

지금 토큰이 이미 저장된 내용을 얼마나 ' 위반 ' 하는가 .

= 손실의 gradient 크기 .

gradient 가 클수록 새롭고 예상 밖 → 더 세계 외운다 .

왜 이게 새로운가

고정 state(벡터 / 행렬) 를 ' 학습되는 신경망 ' 으로 바꾸고 , 무엇을 기억할지를 ' 손실의 gradient ' 가 정하게 했다 . 뒤 논문들이 이 손실 · 규칙 · 구조를 각각 일반화한다 .

write 연산 = 'momentum + weight decay 가 붙은 gradient descent 한 스텝'

master update (η, θ, α 는 상수가 아니라 토큰마다 학습된 gate)

$$u_t = \nabla_M \ell(M_{t-1}; x_t)$$

① surprise = 지금 토큰의 gradient (놀라움)

$$S_t = \eta_t S_{t-1} - \theta_t u_t$$

② momentum = 과거 surprise 누적 (놀란 직후도 기억)

$$M_t = (1 - \alpha_t) M_{t-1} + S_t$$

③ weight decay = α 로 오래된 기억 감쇠 (잊기)

$$y_t = M_t(q_t)$$

read = query q_t 를 갱신된 memory 에 통과

왜 momentum 인가

큰 surprise 직후엔 손실이 이미 낮아 gradient 가 급감 → 이어지는 중요한 토큰이 약하게 저장됨. 과거 surprise 를 buffer 에 누적해 '놀란 시간대 전체' 를 기억.

왜 weight decay 인가

수백만 토큰을 다루면 깊은 memory 도 포화. α_t 배만 남겨 잇는다. 이는 Mamba-2·GLA·Gated DeltaNet 의 gate 를 '임의의 깊은 memory' 로 일반화한 것.

첫 번째 발명

optimizer 의 내부 state(momentum buffer S_t) 를 sequence 레이어의 recurrent state 로 '승격' 시켰다. ablation 기여도 : weight decay > momentum.

A2 와의 정확한 대응

이 세 식은 A2 의 'momentum + weight decay 가 붙은 gradient descent' 와 글자 그대로 같은 모양이다.

다만 대상이 '모델 파라미터' 가 아니라 '추론 중 memory M' 이다 — 학습 (training) 의 도구를 추론 (inference) 안으로 가져왔다.

그 gate 들은 대체 누가 학습하나 ? — 두 개의 loop

바깥 loop = 평범한 pre-training (딱 한 번 , 배포 후 동결)

학습 대상 : key/value/query 투영 , gate 를 뿜는 head, persistent token, attention 블록 , 그리고 memory 의 ' 초기 상태 ' .

도구 : AdamW · 대량 데이터 .

→ 이 전부가 배포 뒤엔 고정 (frozen).

안쪽 loop = memory weights 갱신 (매 토큰 , inference 중에도)

sequence 마다 진화하는 텐서는 딱 둘 : memory weights W_t 와 momentum buffer S_t .

나머지는 전부 동결 .

→ 매 토큰 memory 를 고쳐 쓴다 .

가장 중요한 사실 — outer gradient 가 unroll 된 inner update 를 ' 관통 ' 한다

key 투영을 조금 바꾸면 → 매 토큰 inner gradient 가 바뀌고 → memory 궤적 전체가 바뀌어 → 최종 손실이 바뀐다 . 그 연쇄 전체의 gradient 가 투영을 학습시킨다 . (= 'optimizer 를 미분한다 ')

gate 에 대한 답

$\eta \cdot \beta \cdot \alpha$ 의 ' 값 ' 은 inference 중 매 토큰 새로 계산되지만 , 그 값을 만드는 ' 함수 ' 는 pre-training 에서만 학습되고 이후 동결 . 학습은 함수를 만들고 , inference 는 그 함수로 memory 를 고쳐 쓴다 .

그 결과 — decode 의 정의가 바뀐다

더는 'read-only forward + KV append' 가 아니라 forward + backward (gradient) + optimizer step + read. backward 가 decode 안으로 들어온다 . (→ Part C 에서 이 비용을 쟀다 .)

매 토큰 순차 gradient 는 가속기 최악 → chunkwise 로 GEMM 화

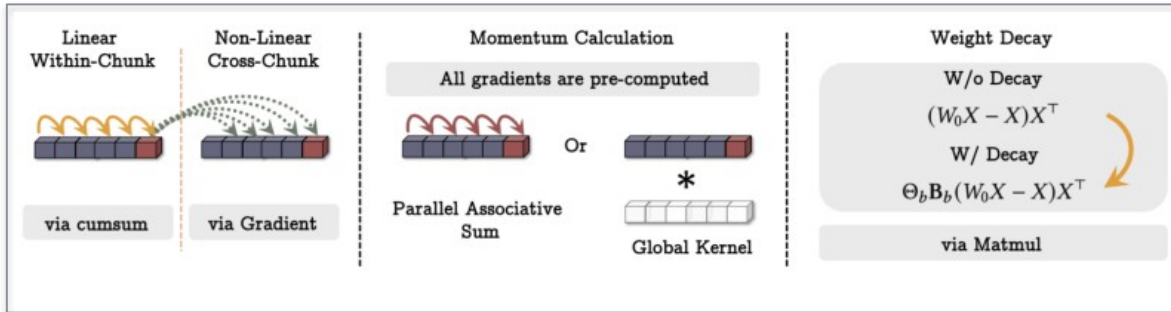


Figure 1 — chunk 안은 병렬, chunk 경계만 비선형 재진입

출처: 2501.00663 Fig.1

핵심 트릭

sequence 를 크기 C 의 chunk 로 자르고, chunk 안 모든 gradient 를 'chunk 시작 시점의 동결된 weights' 에서 한꺼번에 평가 → C 개를 동시에 = GEMM 2 개. C 가 write GEMM 의 내적 차원 = arithmetic intensity 의 손잡이.

세 조각

- chunk 안 갱신은 linear → cumsum, 경계 재진입은 nonlinear → gradient
- momentum buffer = 1 차 선형 recurrence → Mamba 의 selective scan 과 같은 모양 → parallel scan(log-depth)
- weight decay = 누적 곱을 대각으로 접은 matmul 로 흡수

핵심 협상

chunk 안은 linear 라서 병렬, chunk 사이는 nonlinear 라서 표현력. 순차 임계경로가 L 이 아니라 L/C 번의 MLP 적용으로 줄고, 그 비선형 경계가 표현력의 근거. (DeltaNet 이 closed-form 위해 전부 linear 로 남은 것과 의도적으로 다른 선택.)

attention(정밀 단기) + LMM(서서히 잊는 장기) 을 합치는 3 가지

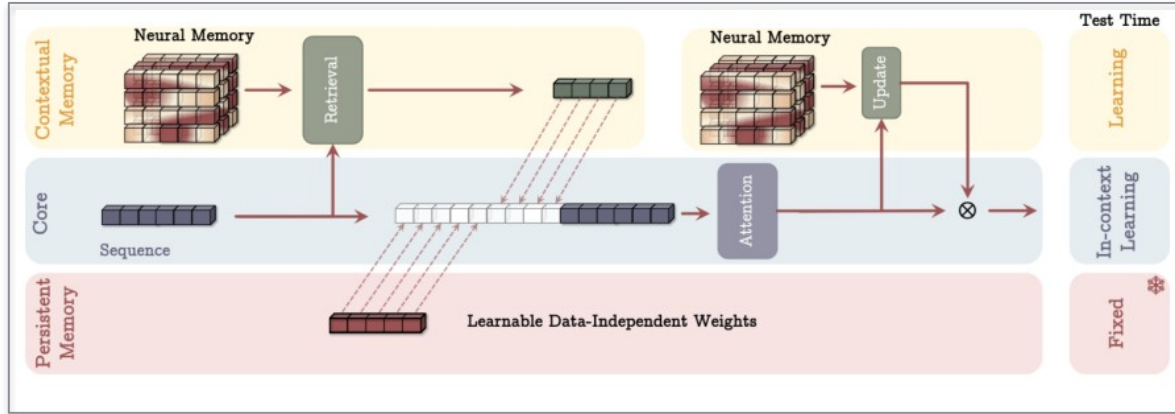


Figure 2 — MAC: 검색한 과거를 현재 앞에 붙여 attention 이 함께 봄
출처 : 2501.00663 Fig.2

세 갈래 (공통)

persistent(입력 독립 · 동결 작업기억) + core(단기 attention · ICL) + long-term memory(위 식으로 test-time 학습).

MAC (Memory as Context) — 가장 강함

contextual 갈래가 과거를 검색해 현재 입력 앞에 붙이고 , attention 이 ' 검색된 역사 + 현재 ' 를 함께 본다 → 장기 정보 필요 여부를 토큰 단위로 판단 . attention 이 write filter 역할 .

MAG · MAL

MAG: sliding-window attention 과 LMM 을 병렬로 돌려 gate 로 섞음 . MAL: LMM 을 attention 앞 레이어로 직렬 — 논문이 ' 가장 약함 ' 이라 명시 (파이프라인 힘이 각 단으로 상한).

숨은 한 방

H3 이래 대부분의 기존 hybrid(Samba · Griffin) 가 사실상 MAL 모양 → ' MAC/MAG > MAL ' 결과는 hybrid 설계 관행 자체에 대한 기소장 .

test-time 역할 분담

memory = 여전히 학습 중 · attention = in-context learning · persistent = 고정 .
(그림의 눈송이 = 동결 .)

forward+backward 를 gate out 까지 · shape 명시 (선형 메모리 기준)

shape: $X \in \mathbb{R}^{C \times d}$ (청크 C 토큰) · $W_Q, W_K, W_V \in \mathbb{R}^{d \times d}$ · 메모리 $M^{\wedge}\{t\} \in \mathbb{R}^{d \times d}$ (청크 시작, frozen) · persistent $P \in \mathbb{R}^{N_p \times d}$ · gate $w_\eta, w_\alpha, w_\theta \in \mathbb{R}^d$

- ① $Q, K, V = XW_Q, XW_K, XW_V \in \mathbb{R}^{C \times d}; [\eta, \alpha, \theta] = \sigma(X[w_\eta, w_\alpha, w_\theta]) \in \mathbb{R}^{C \times 3}$ 병렬 GEMM — gate 산출
- ② $H = Q(M^{(t)})^\top \in \mathbb{R}^{C \times d}$ (pre-update read) 병렬 (read)
- ③ $\tilde{X} = [P; H; X] \in \mathbb{R}^{(N_p+2C) \times d}; y = \text{Attn}(\tilde{X})_{[-C:]} \in \mathbb{R}^{C \times d}$ 병렬 (attention)
- ④ $\hat{K} = yW_K, \hat{V} = yW_V \in \mathbb{R}^{C \times d}; E = \hat{K}(M^{(t)})^\top - \hat{V} \in \mathbb{R}^{C \times d}$ 병렬 — 예측오차 E
- ⑤ $g_i = \nabla_M \frac{1}{2} \|M^{(t)} \hat{k}_i - \hat{v}_i\|^2 = e_i \hat{k}_i^\top; G = (\theta \odot E)^\top \hat{K} \in \mathbb{R}^{d \times d}$ = backward, GEMM 1 회
- ⑥ $S_i = \eta_i S_{i-1} - \theta_i g_i \Rightarrow$ associative scan, depth $\log C, S \in \mathbb{R}^{d \times d}$ 순차 임계경로 (scan)
- ⑦ $M^{(t+1)} = \left(\prod_{i=1}^C (1 - \alpha_i) \right) M^{(t)} + S_C \in \mathbb{R}^{d \times d}$ (decay + update) cumprod→matmul
- ⑧ $o = y \odot (y(M^{(t+1)})^\top) \in \mathbb{R}^{C \times d}$ (gate out) 병렬 — 최종 출력

핵심 — 왜 ' 동시에 ' 되나 (backward 의 병렬화)

C 개 토큰의 gradient(⑤) 를 모두 ' 같은 chunk-start 메모리 $M^{\wedge}\{t\}$ ' 에서 평가하므로 순차 backprop 이 아니라 GEMM 1 회 ($G=(\theta \odot E)^\top \hat{K}$) 로 병렬화 . 유일한 순차 임계경로는 ⑥ momentum scan(깊이 $\log C$). deep(MLP) 메모리도 골격 동일 — 각 층 gradient 를 chunk 시작 가중치에서 batched backprop. (선형 메모리 기준 shape 전개 — 원논문 Fig.1 chunkwise 병렬화의 shape 화 , 일부 추론 .)

state 가 길이에 무관하게 일정 → 초장문에서 훨씬 큰 모델을 이긴다

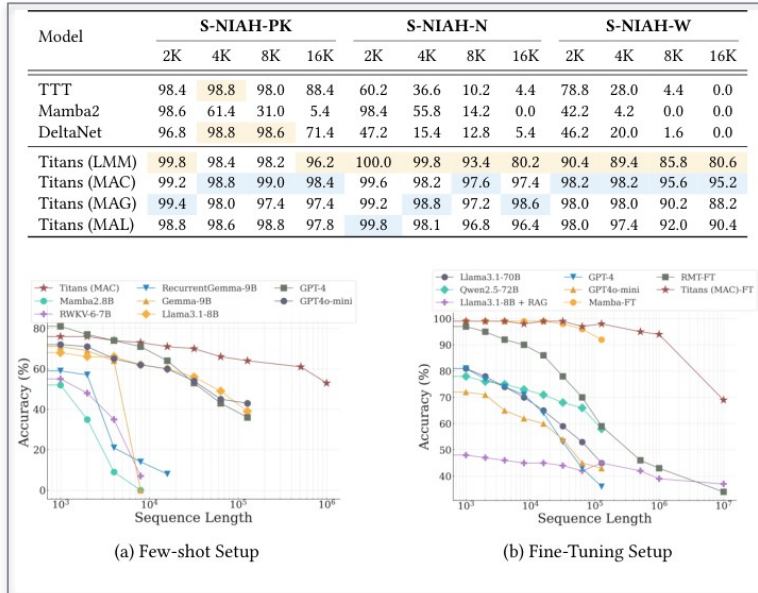


Figure 6 — BABILong: MAC 가 GPT-4 · Mamba 2.8B · RWKV 7B 를 앞섬 (적은 파라미터로)
출처 : 2501.00663 Fig.6

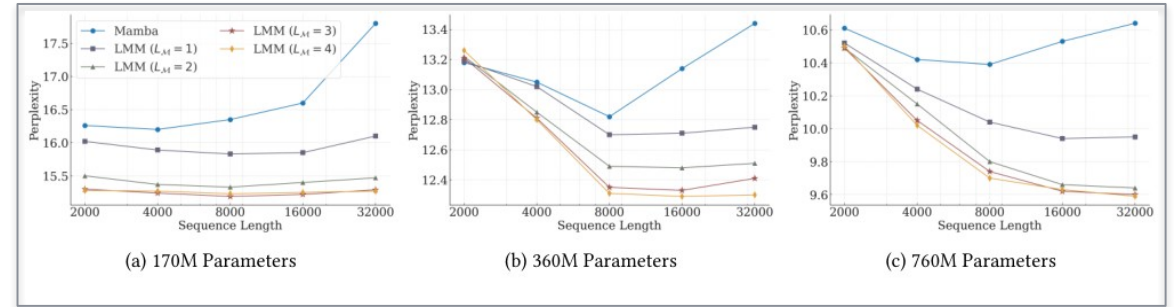


Figure 7 — 메모리 깊이 (1→4 층) ↑ → 모든 길이에서 perplexity ↓
출처 : 2501.00663 Fig.7

읽는 법 (정직성 : 방향 · 비율 위주)

- S-NIAH: Titans 는 2K~16K 전 구간 높은 정확도 . Mamba-2 는 긴 구간 붕괴 (얇은 state), DeltaNet 은 과제별 붕괴 (forget 없음), TTT 는 처짐 (retention gate 없음) — 세 성분이 각 경쟁자를 이긴 이유와 일치 .
- fine-tuning BABILong: 작은 MAC 가 파라미터 ~70 배인 Llama3.1-8B/70B · Qwen2.5-72B 를 넘어 2M 토큰 너머까지 유지 . (비통제 비교라 인과는 ablation 범위에서만 주장 .)

point design 이 분해되어 좌표계가 된다 → Miras

정직한 한계 5 가지

- ① point design — 왜 하필 제곱오차 손실 ? 왜 SGD+momentum+decay? 논거 없음 .
- ② 미명시 세부 — gate 함수형 · memory 폭 · chunk 크기 (v1 부재).
- ③ 근사 비용 미측정 — chunk 안 gradient 는 stale, chunk sweep 없음 .
train(chunk)/serve(토큰) 불일치 제기조차 안 됨 (→ TNT 의 출발점).
- ④ 표현력 정리에 증명 없음 .
- ⑤ 증거가 전부 $\leq 760\text{M} \cdot 4\text{K}$ 훈련길이 . decode wall-clock 수치는 전무 (→ Part C).

시스템 관점 핵심 교환

state 가 길이에 무관하게 일정 (memory weights + momentum buffer = 고정 크기).

KV cache 는 길이에 비례 → 2M 토큰 능력이 정확히 이 교환 위에 섬 . 대가 :
momentum 이 state 2 배 , decode 가 read-modify-write.

다음으로의 연결 — Miras

Appendix C 가 Gated DeltaNet · Longhorn · RWKV-7 · TTT 를 전부 자기 update 의 특수 사례로 ' 회수 ' .

momentum 끄면 Gated DeltaNet, forget+momentum 끄면 TTT ...

→ 계보 전체가 ' 하나의 설계 공간의 점들 ' . 그 좌표축을 명명한 것이 Miras.

한 줄 요약

Titans = memory 를 weights 로 보고 , surprise 를 gradient 로 정의 , momentum 으로 시간 흐름 , weight decay 로 잊기 , 깊은 MLP 에 넣고도 chunkwise 로 병렬 학습 .
남은 질문 (왜 이 손실 ? 왜 이 규칙 ?) 이 Miras · Atlas · HOPE 로 이어진다 .

Miras — 메모리는 하나의 online 최적화 ; 4 축 설계공간

핵심 발명 / 주장

1

sequence layer = 하나의 online 최적화 문제로 통일해 볼 수 있다

Titans · DeltaNet · linear attention 등은 모두 '어떤 목적을 어떤 규칙으로 최소화하나'의 특수해

2

설계는 4 개의 축으로 분해된다 : attentional bias · retention · architecture · algorithm

① 무엇을 기억 (손실) ② 무엇을 잊기 (retention) ③ 메모리 구조 ④ 갱신 알고리즘

3

이 축에서 새 인스턴스를 만든다 : Moneta · Yaad · Memora (FTRL · Learning-Retaining)

발명이라기보다 '설계 공간의 존재 증명 + 각 축의 효과를 실험으로 확인'

논거 흐름

Titans 재해석 (surprise=attentional bias, forget=retention) → 축을 일반화 → 손실을 L2 에서 Lp/robust 로 , retention 을 다양하게 → 새 조합
Moneta/Yaad/Memora → 축별 ablation

증명 (뒤에서 논문 그림 · 표로)

모델 크기 · 문맥 길이 스케일링 (Fig.3), 손실차수 p · retention q 의 효과 (Fig.4) 로 각 축이 실제로 성능을 가른다

Titans 는 한 점만 찍고 끝났다 — 왜 L2? 왜 momentum? forgetting 이 맞나 ?

문제 — 특수 사례는 보였는데 '일반형의 좌표축' 이 공백

Titans 는 존재 증명이었지만 3 가지에 답하지 않았다 : ① 왜 L2 loss(내부 목적함수)? ② 왜 momentum·weight decay(딥러닝 관행을 그냥 이식)? ③ forgetting 이 옳은 개념인가 ?

Titans 스스로 'L2 너머의 목적함수' 와 '더 나은 optimizer' 를 open problem 으로 남겼다 .

관찰 1 — 전부 associative memory

거의 모든 시퀀스 모델 = key 를 넣으면 value 를 돌려주는 연상 메모리 .

내부 목적함수는 단 2 종뿐이었다 : dot-product 유사도 / L2 regression.

이 내부 loss 를 attentional bias 로 명명 = '이 메모리가 무엇을 우선해서 기억하는가' !

관찰 2 — forgetting 은 없다 , retention 만 있다

기존 forget gate = 전부 L2 정규화의 특수형 .

하는 일은 '지우기' 가 아니라 '새것 배우기 vs 이전 상태 머무르기' 의 저울질 — 모델은 유지하지 않기로 결정할 뿐 .

→ forget gate 를 retention gate 로 개명 .

→ 이어지는 결론

이 두 관찰을 밀고 나가면 시퀀스 모델 설계가 4 개의 독립적 축으로 분해된다 . '시퀀스 모델 = 무언가를 최소화하는 연상 메모리' .

메모리를 4 개의 독립적 '선택' 으로 분해 — retention 이 결정적 레버

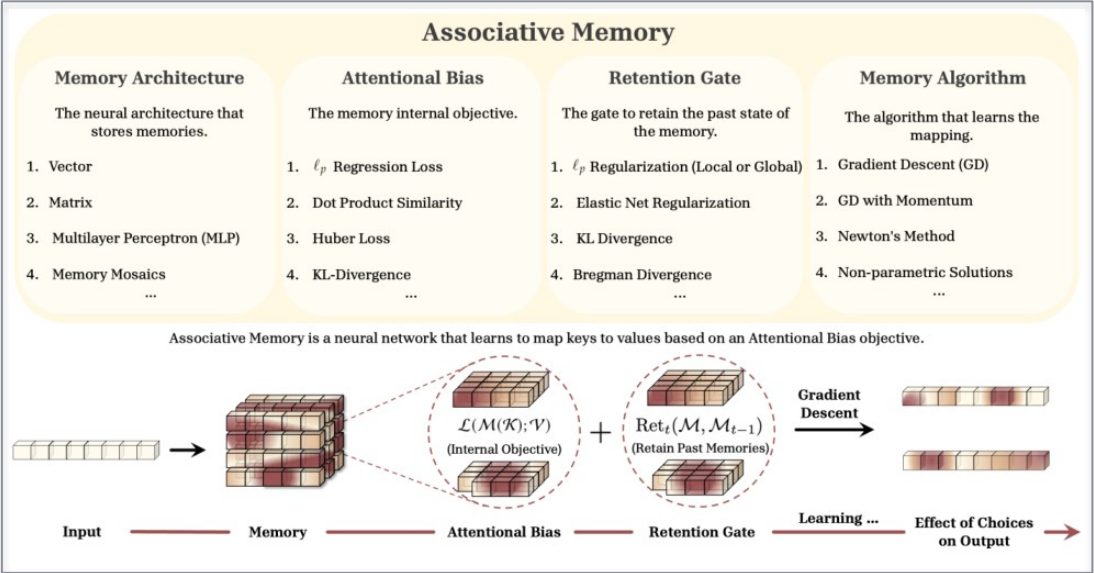


Figure 1 — 매 token 최소화되는 안쪽 목적 = attentional bias + retention, GD 로 푼다
출처 : 2504.13173 Fig.1

① Memory architecture

메모리 구조 — vector · matrix · deep MLP.

② Attentional bias

무엇을 우선해 기억 ? = 내부 목적함수 L. dot-product · L2 · Lp · robust.

③ Retention gate

새것 배우기 vs 옛것 유지의 균형 . long-context 를 좌우하는 결정적 레버 .

④ Memory learning algorithm

그 목적을 어떻게 푸냐 — GD · momentum · Newton · closed-form.

의의

Titans 를 특수점으로 품는 지도 . ' 무엇을 바꾸면 무엇이 좋아지나 ' 를 조직적으로 탐색 가능하게 했다 . 이후 모든 논문이 이 어휘로 자기 위치를 설명한다 .

attentional bias 하나만 바뀌도 기존 모델들이 재유도된다

bias = dot-product → '덧쓰기'

'비슷한 key 에 비례해 크게 써라' → 순수하게 더하기만 하는 write.
= linear attention · RetNet · Mamba-2 (옛 Hebbian 덧쓰기).

bias = L2 regression → '고쳐쓰기'

'이미 있던 걸 고쳐 써라' → 예측오차만큼 수정.
= DeltaNet · Gated DeltaNet · RWKV-7. (A3 의 '덧쓰기→고쳐쓰기'가 '목적함수만 바꿨을 뿐'으로 압축.)

softmax attention 도 이 공간의 한 점

L2 regression 을 '압축 없이 비모수적'으로 풀 극한 → retention 이 없다 → state(KV cache) 가 계속 커진다.
attention 의 메모리가 길이에 선형으로 느는 이유 = '압축하지 않는 연상 메모리이기 때문'으로 설명된다.

한 문장 정리 + 정직성

덧쓰기 (linear) · 고쳐쓰기 (delta) · 안 압축 (softmax) 이 전부 'attentional bias 축의 세 좌표'로 통일된다.

다만 이 표의 등호는 '원 설계 방정식' 수준의 동일시다. retention 열의 L2 는 세부가 다른 gate 들을 뭉뚱그린 것 — 분류학으로 정확하되 구현 완전등가로 읽으면 과독 (논문 각주도 이 해상도 한계를 인정).

bias·retention 축에 새 선택지를 넣다 (optimizer 는 일부러 plain GD)

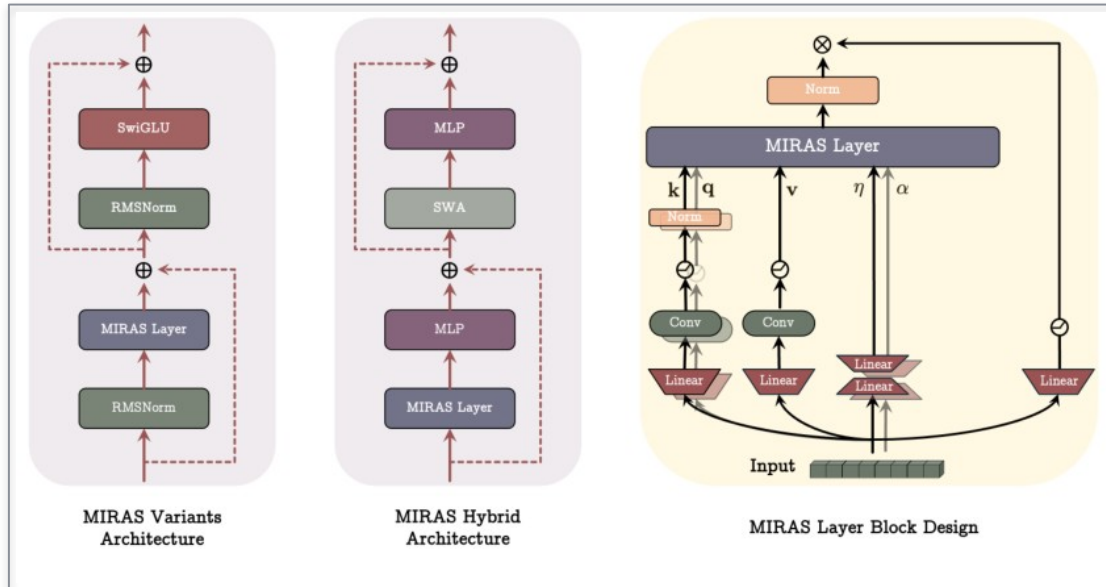


Figure 2 — Moneta/Yaad/Memora: recurrent · SWA hybrid · layer 설계
출처 : 2504.13173 Fig.2

Moneta — Lp loss

Lp(p=3) 로 잘 안 떠오르는 '놀라운 token' 에 L2 보다 날카로운 기억 압력 + Lq(q=4) 정규화로 메모리 norm 을 통제된 껍질에 붙잡음 .

Yaad — Huber loss (robust)

오차가 학습된 threshold 보다 크면 (=outlier token) 그 크기를 깎아 기억 → 노이즈 · 적대 구간이 자기 크기만큼 메모리를 흔들지 못한다 .

Memora — KL retention (가장 우아)

retention 을 KL divergence 로 → 메모리를 확률 simplex 에 가둠 . 갱신이 softmax 형태라 state 가 항상 양수 · 합 =1.

→ 문맥이 아무리 길어도 state 가 발산 불가 (retention-as-renormalization).

실험 설계 의도

셋 다 optimizer 는 plain GD(momentum 일부러 제거) — expressivity 를 optimizer 가 아니라 ' 목적함수와 retention' 에 싣고 그 효과만 보려는 통제 .

attention 한 layer 도 안 쓰고 attention-hybrid 를 이긴다

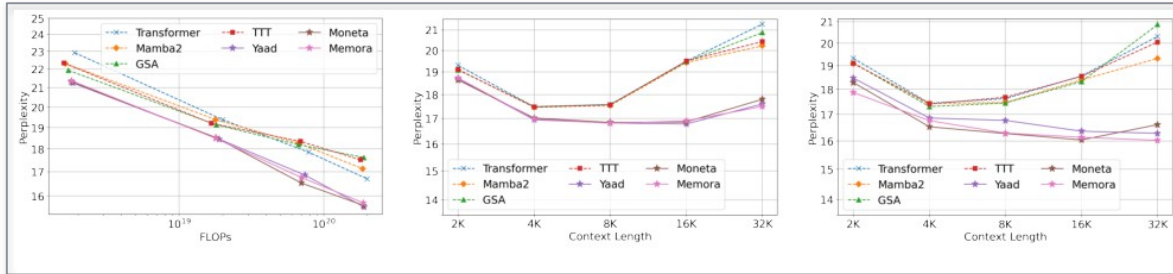


Figure 3 — scaling: baseline 은 16K 넘으면 perplexity 도로 상승 , 세 변형은 완만 유지

출처 : 2504.13173 Fig.3

헤드라인

더 나은 bias+retention 이면 attention 없이도 hybrid 를 이긴다 . 1.3B/100B token 에서 순수 recurrent Yaad 가 Gated DeltaNet 은 물론 hybrid Samba · GDN-H2 까지 perplexity 에서 이겼다 .

ablation 의 교훈

p(bias) 를 바꾸면 성능은 오르내리나 scaling 모양은 불변 . q(retention) 를 바꾸면 scaling 패턴 자체가 바뀐다 → 기여순위 retention > deep memory > bias.

'retention 이 결정적 레버' 의 직접 증거 + 정직한 한계

위 ablation 이 관찰 2(retention 이 핵심) 의 가장 직접적 증거 . 한계 : 실증이 1.3B · 100B token 에서 멈춘다 . 7B+ · RLHF 후 · exact-copy recall 실무에서도 순서가 유지되는지는 미답 , 효율 증거도 FLOPs 곡선 하나뿐 (wall-clock 없음) .

진짜 기여는 세 모델이 아니라 좌표계 → Atlas 가 빈 축을 채우러 간다

진짜 기여 = 좌표계

Moneta/Yaad/Memora 수치는 1.3B 에서 멈추지만 , attentional bias·retention gate·4 축이라는 ' 어휘 ' 는 이후 모든 논문이 무상으로 가져다 쓴다 . Titans= 존재 증명 , Miras= 그 move 가 연 공간의 좌표계 .

Miras 가 다음 논문에 넘긴 숙제 3 가지 = Atlas 의 목차

- ① 비어 있는 optimizer 축 — 세 모델 다 plain GD(momentum 제거) . 더 나은 optimizer(Titans momentum·Newton 법) 가 새 bias·gate 와 합쳐지면 ? → Atlas 는 inner loop 에 Muon 을 심는다 .
- ② token 단위 목적함수 — 목적함수가 여전히 'token 하나 ' 만 본다 . 최근 여러 token 이 함께 잘 저장됐는지 아무도 안 물어 . → Atlas 는 Omega rule(최근 여러 token 함께 최적화) 을 도입 .
- ③ capacity 이론 부재 — online 최적화 도구를 통째로 수입했지만 그 보증이 요구하는 convexity 를 2-layer MLP 가 깬다 . → Atlas 는 정식 capacity 이론을 붙인다 .

한 줄

Miras 가 지도를 그렸다면 , Atlas 는 그 지도의 ' 빈 축 ' 을 최적화 이론으로 채우러 간다 .

Atlas — 토큰이 아니라 ' 문맥 ' 을 기억하고 , 용량을 키운다

핵심 발명 / 주장

1

Omega rule: 개별 토큰이 아니라 sliding-window ' 문맥 전체 ' 를 한 번에 기억

$y_{\{t,i\}}$ 가중으로 최근 c 개 토큰을 함께 최적화 — 'learning to memorize the context'

2

feature map ϕ_p 로 메모리 용량을 matrix 한계 ($\sqrt{\text{params}}$) 너머 d^p 까지 확장

polynomial feature map 으로 표현력 $\uparrow$ (softmax 는 사실상 무한 용량의 극단)

3

메모리 학습을 GD 가 아니라 Muon(근사 2 차) optimizer 로

곡률 정보를 써 더 잘 · 빠르게 메모리를 적합 — A2 의 optimizer 발전을 메모리 학습에 적용

논거 흐름

메모리 용량이 병목 (Titans Fig.7 의 연장) $\rightarrow$ Hopfield 용량 이론 $\rightarrow$ feature map 투영으로 확장 $\rightarrow$ 문맥 단위 목적 (Omega) $\rightarrow$ 최적화도 Muon 으로 $\rightarrow$ DeepTransformers/Dot 변형

증명 (뒤에서 논문 그림 · 표로)

BABILong 에서 Titans 초과 (Fig.4), associative recall 용량 실험 (Fig.7), 문맥 · FLOPs 스케일링 (Fig.8)

목표 자체를 바꾼다 : ' 이 token 하나 ' → ' 최근 문맥 전체를 함께 '

관점 — Miras 가 비워둔 optimizer 축 포함, 세 구멍을 한 논문에서 메운다

무엇을 목표로 삼는가 (objective) · 얼마나 담을 수 있는가 (capacity) · 얼마나 잘 채우는가 (optimizer).

① online 갱신 (근시안)

memory 가 매 step 현재 token 하나만 보고 최적화,
과거는 retention gate 로 흐릿하게 유지될 뿐.

→ token 단위 greedy 암기. ' 문맥 전체가 잘
저장됐나 ' 는 아무도 안 묻는다.

② 용량 한계

단순 행렬 memory 는 파라미터를 아무리 많이 줘도,
저장 가능한 연상 ($k \rightarrow v$) 수가 key 차원 d_k 수준에서
막힌다.

→ state 를 키워 파라미터를 더 쓰는 것 ≠ 용량을 늘리는
것.

③ 관리 빈약

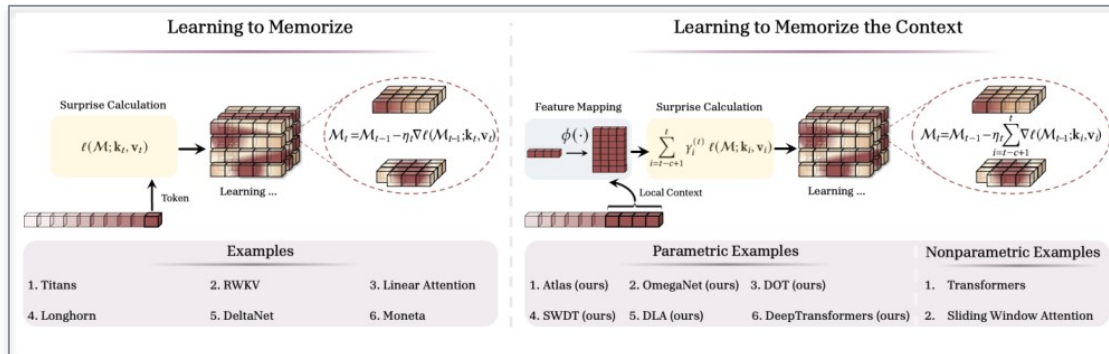
inner optimizer 가 거의 전부 1 차 gradient descent.

→ loss 표면의 나쁜 local minimum 에 앉으면 질
낮은 $k \rightarrow v$ 매핑을 그대로 저장.

세 처방 = 부제의 세 단어

Omega rule(목표를 window 로 넓힘) · feature map(용량 상한을 옮김) · Muon(그 상한 안에서 도달 품질을 올림). 이 셋의 조합이 이 섹션 전체.

context 를 기억한다 , 그리고 용량을 정리로 정의한다



핵심 전환

여러 token 에 걸친 사건은 어느 하나도 개별로는 놀랍지 않지만 묶음으로는 중요할 수 있다 ('어제 김대리가 예산 승인'). token 단위 loss 는 이를 구조적으로 놓친다 → 최근 c 개를 함께 최적화 .

용량 이론 (이 게보 최초)

Prop 1: 행렬 memory W 는 GD 로 최적화해도 최대 $O(d_k)$ 개만 정확 저장 (순수 rank 논증).

Prop 2: polynomial feature map ϕ_p 로 차원을 $\Theta(d_k^p)$ 로 → 용량 $O(d_k^p)$, 어떤 optimizer 로도 이 상한을 못 넘는다 .

Figure 1 — token 하나 (원) vs 최근 c 개 함께 (오 , Omega). 오른쪽 끝 = attention 도 windowed regression

출처 : 2505.23735 Fig.1

함의 — attention 이 이기는 이유가 정리로 설명된다

무한 차원 feature(Kronecker self-tensoring) 를 만들면 $\exp(\mathbf{q} \cdot \mathbf{k}) = \phi^*(\mathbf{q}) \cdot \phi^*(\mathbf{k}) \rightarrow \text{softmax attention} = \text{무한 차원 위 associative memory} = \text{용량 무한}$. attention 이 긴 문맥 recall 에서 고정 state 를 이기는 건 신비가 아니라 '용량 상한의 차이' . (선형독립 key 정확 보간이라는 이상화 기준 → 근사 대리지표로 읽기 .)

delta rule 을 window 크기만큼 rank 로 일반화 (c=1 이면 Titans)

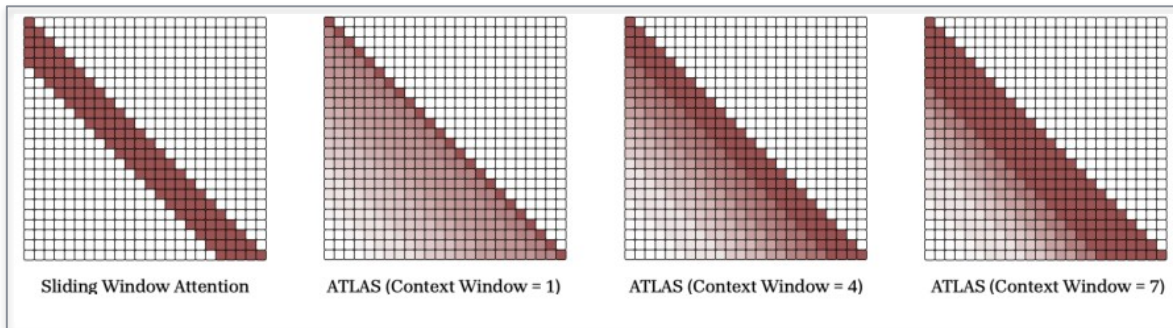


Figure 2 — SWA(비모수) vs Atlas: window c=1,4,7 로 키우면 의존이 하삼각 전체로 번짐
출처 : 2505.23735 Fig.2

$$\min_M \sum_{i=t-c+1}^t \gamma_{t,i} \|M(k_i) - v_i\|^2$$

기호

$\gamma_{\{t,i\}}$ = window gate. 0 이면 그 token 을 최적화에서 hard pruning(admission control), 1 이면 온전히 포함 .
c = window 길이 . sliding window 라 step 당 gate 수가 c 개로 상수 → recurrent 장점 유지 .

대수적 정체

DeltaNet 전이 = key 1 개짜리 rank-1 수정 . Omega rule = c 개 key 합 = rank-c 수정 .
c=1 로 두면 정확히 Titans (Titans = Omega 의 window-1 특수 사례).

왜 global 이 아니라 window 인가

문맥 전체 global 최적화는 (i) 매 step 모든 과거 key/value 를 들고 있어야 해 고정 state 존재 이유가 사라지고 (ii) 무관 구간을 잘라낼 gate 가 없다 . Omega 는 sliding window 로 둘 다 해결 (γ 가 pruning gate).

근사 2 차 관리 , 그리고 같은 기계로 Transformer 재유도

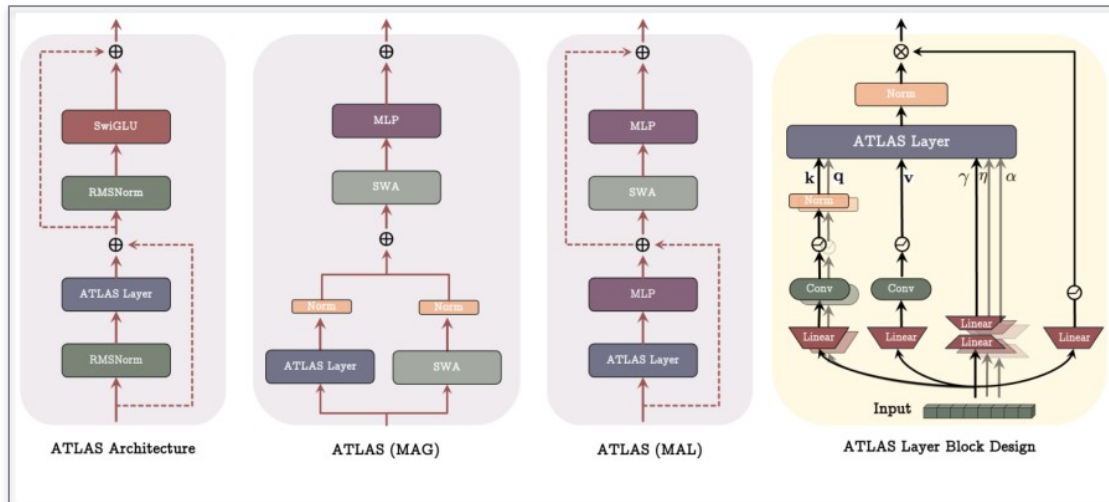


Figure 3 — Atlas Layer 배선 + hybrid(MAG/MAL). Atlas/Atlas++/OmegaNet, DeepTransformers·Dot

출처: 2505.23735 Fig.3

Muon inner optimizer (③ 관리)

momentum buffer S 에 windowed gradient 를 쌓고 Newton-Schulz 반복을 κ 번 $\rightarrow$ semi-orthogonal(UV^T) 로 수렴 $\rightarrow$ update singular value 균등화 = 2 차 정보 근사.

κ = NS 반복수 = 'internal test-time compute' dial. 더 돌리면 품질 $\uparrow$ ·inference FLOPs $\uparrow$ (state 안 건드리고 연산 $\leftrightarrow$ 품질 교환). 실전 $\kappa=5$.

부산물 — Transformer 두 일반화 가족

attention = Nadaraya-Watson kernel regression 의 비모수해로 재서술.

feature map 을 정확한 $\exp \phi^*$ 로 $\rightarrow$ DeepTransformers(unnormalized softmax 의 strict 일반화). 거기에 Omega rule $\rightarrow$ Dot = 'error-correcting attention'.

병렬화 유지

window 는 banded mask 하나로 처리 (각 token gradient 1 회 계산 + mask 합산) $\rightarrow$ window 는 훈련 비용 거의 안 바꾸는 품질 knob. Muon 도 chunk 경계 상태에서 gradient 를 평가하면 momentum 이 linear scan 으로 분리되고 NS-5 는 batched matmul.

초장문 외삽은 정점 — 그러나 Muon 의 실증 가치는 논쟁적

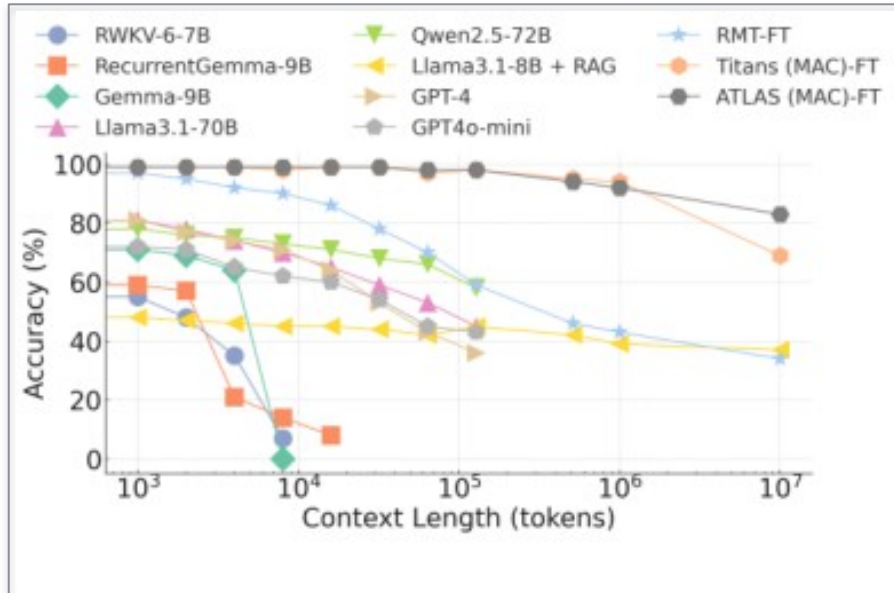


Figure 4 — BABILong: 4K 훈련으로 10M 까지 외삽, Titans 붕괴 지점에서도 유지
출처: 2505.23735 Fig.4

결과 (방향)

BABILong: 1M 까지 Titans 동급, 10M(Titans 붕괴) 에서도 높은 정확도 — 훈련 문맥의 수천 배 외삽. LM·commonsense 에서 Atlas/Atlas++ 가 Titans·GDN·Transformer++ 상회 (순수 recurrent 최고 그룹).

불편한 진실 (한계)

- ① in-context retrieval 은 여전히 Transformer 우위 — gap 좁혔으나 못 닫음 (용량 이론이 예측한 방향).
- ② ablation: window·feature map·deep memory 는 명확히 기여하나 Muon 제거 시 perplexity 가 오히려 개선. Muon 이 산 건 reasoning 0.21 점뿐 → 'optimally' 는 대부분 window·capacity 의 공.
- ③ **κ 품질 곡선 측정 없음.** ④ **wall-clock 수치 전무 (전부 구조 논증).**

'무엇을 얼마나 잘 기억하는가'의 정점 → 이제 '얼마가 드는가'(TNT)

정리

세 구멍을 세 손잡이로 : Ω (목표) · feature map(용량) · μ (관리). 부산물로 softmax 를 무한용량 associative memory 로 정식화하고 Transformer 두 일반화 가족 (DeepTransformers · Dot) 을 파생 . objective · capacity · optimizer · 이론까지 다 갖춘 정점 .

다음으로의 연결 — TNT

Atlas 의 모든 것은 chunkwise 트릭 위에 서 있다 — gradient 를 'chunk 시작 상태 ' 라는 stale snapshot 에서 평가하는 근사 .

그 근사의 오차는 정량화된 적이 없고 , chunk 크기 C 는 그냥 throughput knob 으로만 취급됐다 (사실 C 는 ' 계산되는 함수 자체 ' 를 바꾸는 semantic knob).

특히 치명적 mismatch 가 방치 : 훈련은 큰 C 에서 , decode 는 $C=1$ 의 세계 . → TNT 가 정확히 여기서 시작해 chunk 경제학 전체를 재설계한다 .

조용한 복선

Atlas 가 'test-time training' 을 'test-time memorization' 으로 개명하며 'in-context 적응은 학습이 아니다 ' 라고 선을 그은 순간 , ' 그럼 진짜 continual learning 은 어디서 ? ' 가 미결로 남는다 → Nested Learning · Sleep.

TNT — 학습 레시피와 serving 구조를 함께 설계

핵심 발명 / 주장

1

train chunk 와 serve chunk 가 다르면 품질이 붕괴한다 (핵심 관찰)

C=64 로 학습한 550M Titans 를 chunk=1 로 서빙하면 성능 급락 — 학습 / 추론 불일치

2

해법 : train chunk(큰) 와 serve chunk(작은) 를 decouple 하고 2-stage 로 정렬

큰 chunk 로 싸게 병렬 학습 → 2 단계로 작은 serve chunk 에 맞춰 재정렬

3

hierarchical memory: global(prefill, 병렬) / local(decode, 세밀)

긴 문맥은 global 로 한 번에 , 생성은 local 로 토큰별 — 병렬성과 정밀도를 모두

4

Q-K projection 추가로 메모리 읽기 / 쓰기 표현력 보강

chunk 정렬과 함께 serving 품질을 끌어올리는 보조 장치

논거 흐름

deep memory 를 큰 chunk 로 싸게 훈련 → 그런데 serve chunk 가 다르면 붕괴 (2.6 배 악화) → 2-stage 로 정렬 → 계층 메모리로 병렬 + 세밀 → serving 구조 정의

증명 (뒤에서 논문 그림 · 표로)

chunk sweep 의 V자 곡선 (Fig.2), 최대 17 배 속도 (Fig.4/5), 품질 표로 train/serve 정렬 효과 입증

chunk 크기 하나가 품질과 속도를 동시에 결정한다 (= 훈련 경제학)

chunkwise-parallel training의 트레이드오프

C 개 token 을 묶어 'chunk 시작 상태' 에서 모든 gradient 를 한꺼번에 계산 (GPU 가 놀지 않게).

C ↑ → matmul 커져 하드웨어 잘 뚫, 그러나 gradient 가 stale(낡음) → 근사 품질 ↓ . C ↓ → gradient 신선하나 연산이 잘게 쪼개져 하드웨어가 논다 (작은 chunk = peak 대비 5~10% 미만 = MFU 한 자릿수).

Challenge 1 — 비선형 recurrence 는 병렬화가 안 된다

deep memory 는 fine-grained 위해 작은 chunk 를 원하는데 그럼 memory-bound.
linear attn 는 SRAM 상주 chunk kernel 로 회피하나, 그 kernel 은 '선형 상태 전이' 에 의존. MLP+LayerNorm 비선형 recurrence 엔 이식 불가 (parallel scan 불가).

Challenge 2 — write/read 도메인 shift

write 는 $k \rightarrow v$ 로 학습되는데 읽을 땐 query q 로 읽는다 → 학습 함수의 입력 domain 밖에서 평가 → retrieval 품질 ↓ .
attention 엔 이 문제가 구조적으로 없다 ($q \cdot$ 모든 k 내적을 명시적으로). 압축 memory 로 넘어올 때만 생기는 세금 .

Challenge 3 — chunk-size mismatch (다음 슬라이드에서 그림으로)

훈련 chunk 와 서빙 chunk 가 달라지면 품질이 무너진다 .

핵심 가설 (한 문장)

훈련 효율과 inference 성능을 '하나의 chunk 크기' 가 동시에 결정하게 놔두지 말고, 두 단계로 분리 (decouple) 하라 . TNT = 'Titans iNside Titans' — memory 안에 memory 를 중첩 .

새 실증 발견 : 비대칭 V자 절벽 (훈련 해상도에 over-specialize)

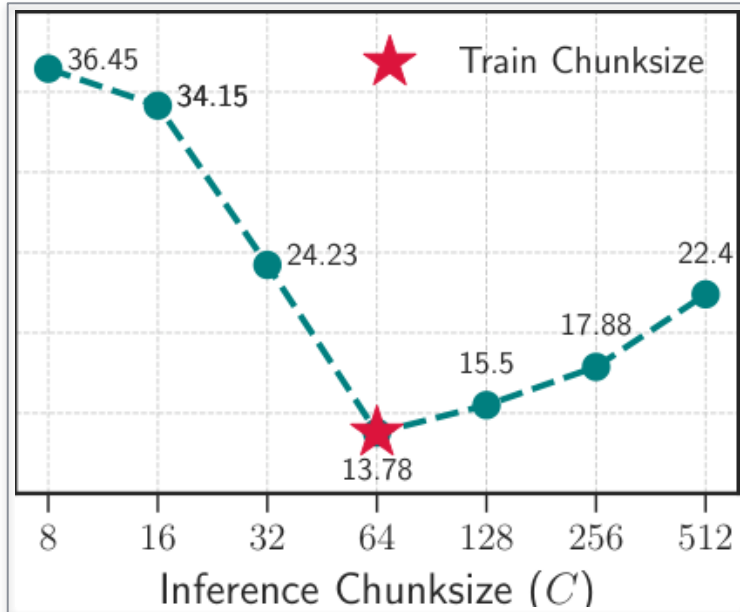


Figure 2 — 550M Titans(C=64 로 pre-train), inference chunk 만 바꿔 측정

출처 : 2511.07343 Fig.2

V자 곡선

바닥이 정확히 훈련 때 쓴 C=64 에 걸림 (ppl 13.78, 최적). 양쪽으로 벗어나면 급격히 악화.
충격: 더 작은 chunk = 더 신선한 gradient = 더 좋아야 상식적. 그런데 C=8 에서 ppl 36.45 로 2.6 배 폭발.

왜 치명적인가

이상적 서빙 = decode 에서 chunk=1(매 token 온라인 갱신).

큰 chunk 로 싸게 훈련한 모델은 chunk=1 근처에서 품질이 무너진다 → 그 이상적 서빙으로 직행할 수가 없다.

읽는 법 (정직성)

이 그림은 TNT 원저자 Figure 2 이고 우리 스터디 실측이 아니다. 절대 수치보다 '비대칭 V자 절벽'이라는 방향성을 기억. (Titans 가 제기조차 안 했던 train/serve 불일치가 여기서 정면으로 드러난다.)

reset 이 병렬화 불가능한 비선형 사슬을 끊는다

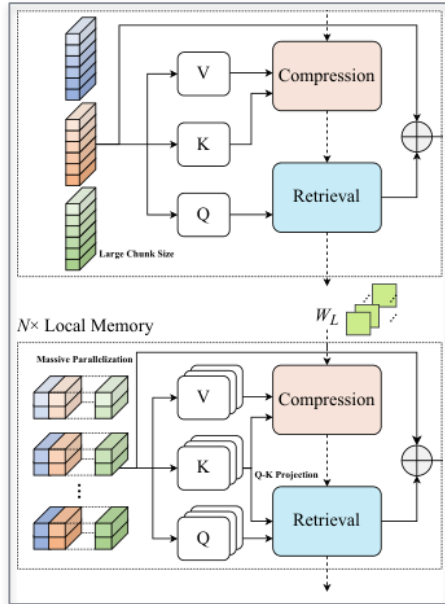


Figure 3 — Stage 1: global(위, 큰 chunk 순차) + N 개 local(아래, reset · 병렬). Q-K 는 local에만

출처: 2511.07343 Fig.3

역할 분담

global 1 개: 큰 chunk(2048)로 sequence 전체 순차 관통 → long-range 보존, 드물고 큰 dense matmul = compute-bound (16K에 8번 handoff).

local N 개: 학습된 초기상태에서 주기적 reset → 대량 병렬, fine-grained 담당.

reset 이 핵심인 이유

비선형 recurrence는 parallel scan 불가 (결합법칙 없음). reset이 그 사슬을 끊어 각 shard가 독립 → 여러 장치 · batch 축으로 병렬.

대가: local은 shard 경계에서 다 잊음 → global이 보전 (ablation: global 빼면 21→25.6 붕괴).

W_init의 승격

reset 지점이 0이 아니라 '학습된' 초기상태 W_init(meta-learn된 prior). Titans에서 암묵적이던 초기상태가 reset을 살아남게 하는 핵심 부품으로.

한 줄

reset = sequence 축을 잘라 '진짜 batch 축'으로 되돌리는 연산. 병렬성을 사기 위해 local의 기억을 주기적으로 태우고, 그 보함을 저해상도 global에 든다.

train-big / serve-small: chunk 를 두 독립 knob 으로 분리

Q-K projection (Challenge 2 처방)

query 를 그대로 memory 에 넣지 말고, 지금까지 관측된 key 들이 스캔하는 부분공간으로 사영한 뒤 넣는다.

사영 행렬 $\Pi_t = \Sigma k \cdot k^T$ 를 매 token rank-1 로 누적 (과거 key 저장 불필요, 상수 크기). = KV cache append 의 rank-1 GEMM 대응물, $\Pi \cdot q$ 는 mat-vec 한 번. domain shift 에 민감한 local 에만 붙고 global 은 raw query.

Stage 2 (Challenge 3 처방)

Stage1(큰 chunk) 로 싸게 pre-train 끝낸 뒤, 작은 local chunk 로 짧게 fine-tune.

놀라운 관찰: 이 짧은 fine-tuning 이 mismatch 를 교정할 뿐 아니라 원래 성능을 넘어선다. 비용은 pre-training 의 5~8% 뿐.

train-big / serve-small 레시피

이상 목표 chunk=1 → autoregressive 서빙과 정확히 맞물림.

global 이 큰 chunk dense 연산으로 prompt 흡수 (prefill), Stage2 로 적응된 local 이 생성 중 token 마다 갱신 (decode).

핵심 성취

chunk 크기가 더 이상 '하나의 타협값' 이 아니라, Stage1 의 throughput knob 과 Stage2 의 해상도 knob 이라는 서로 독립인 두 knob 이 된다. '싸게 학습 + 정확한 서빙' 이 동시에 가능.

같은 chunk 끼리도 7.7 배 → 이득의 원천은 구조 자체

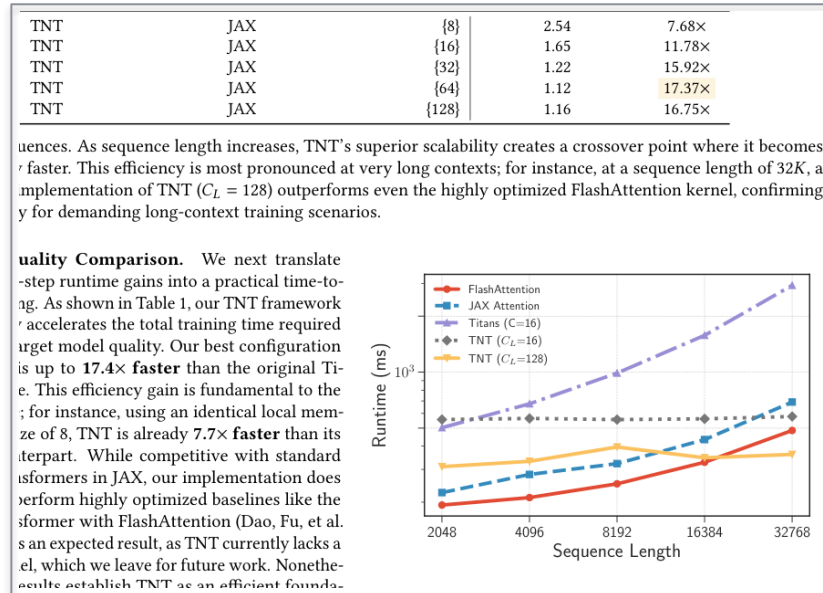


Figure 4 — runtime: 순수 JAX 인데 32K 에서 FlashAttention 을 step 당 이김
출처 : 2511.07343 Fig.4

kernel 이전에 구조가 승부를 갈랐다

custom kernel 최적화된 Gated Transformer 는 아직 못 이기지만 , TNT 는 custom kernel 없는 순수 JAX 인데도 32K 에서 FlashAttention 을 step 당 이겼다 .

속도 · 품질

목표 loss 3.20 도달 : Titans(C=8) ~19.5h vs TNT ~1.1h = 17.37 배 . 같은 chunk 8 끼리도 7.7 배 → 원천은 chunk 키움이 아니라 구조 .

150M/10B: Stage1 만으로 모든 RNN baseline+vanilla Transformer ppl 이김 , Stage2 가 더 내림 . reasoning 은 Gated Transformer 까지 .

ablation

local 1→4 개 ppl 단조 개선 (20.15). global 빼면 25.6 붕괴 . Q-K projection 빼면 ~1 ppl 손해 . 역할 분담이 실증됨 .

정직한 한계

모든 검증이 momentum·gating·Muon 을 제거한 '단순화 Titans' 위 , 150M/10B(이 라인 최소 규모) . 분류 모델과의 합성은 미측정 .

이미 세 개의 update frequency 로 도는 시스템 → Nested Learning 의 씨앗

TNT 가 base 4 부작에 남긴 것

이 계열이 대규모로 갈 수 있도록 '훈련 경제학의 바닥' 을 깔았다. TNT 는 표현력이 아니라 훈련 경제학을 푼 systems 편.

결정적 관념 — 이미 세 개의 update frequency

global 은 2048 token 마다, local 은 매 token 마다, W_{init} ·slow weights 는 훈련에서만 갱신된다.
그리고 느린 시간 스케일에 주차된 지식 ($global \cdot W_{init}$) 이 빠른 스케일의 reset 에서 살아남는다.

다음으로의 연결 — Nested Learning

다만 TNT 에게 이걸 그냥 공학적 방편이었다 — '왜 이게 모델 전체의 조직 원리여야 하는지' 는 말하지 않았다.

다음 논문 Nested Learning 이 그 선언을 한다: 모델과 훈련 절차 전체가 '각자의 update frequency 로 자기 context 를 압축하는 중첩 optimization 문제들의 시스템'!

TNT 의 global/local 이분 → CMS 의 주파수 연속체로, TNT 의 reset → NL 의 re-initialization 으로, 하중을 받게 된 W_{init} → 다섯 knowledge-transfer 기제 중 하나로 분류된다.

여기까지가 base 4 부작

이제 집중해서 볼 두 논문 (HOPE·Sleep) 으로 들어간다.

HOPE — 모든 부품이 '어떤 주파수의 메모리' 인 아키텍처

핵심 발명 / 주장

1

Expressive Optimizers: optimizer 는 gradient 를 압축하는 associative memory 다
momentum·Adam·Muon = 메모리의 특수형 → 더 표현력 큰 optimizer(DGD·M3) 로 승격

2

Self-Modifying Titans: 투영 · 게이트까지 메모리로 만들어 자기 갱신 규칙을 스스로 수정
6 개 메모리 {k,v,q,η,α,mem} 중 q 만 static, 나머지는 test-time 에 갱신

3

CMS(Continuum Memory System): long/short 를 '연속 주파수' 스펙트럼으로 일반화
여러 주파수의 MLP 블록을 이어 붙임 — TNT 의 global/local 을 연속체로 확장 . 합 = HOPE

논거 흐름

backprop 도 self-referential memory → optimizer 도 memory → 그림 architecture 와 optimizer 는 같은 것의 다른 레벨 → 레벨을 더 쌓자 (higher-order ICL) → self-mod + CMS = HOPE. ICL 은 창발이 아니라 ≥ 2 레벨의 structural 결과

증명 (뒤에서 논문 그림 · 표로)

760M/1.3B 벤치 (Transformer++·RWKV7·DeltaNet·Titans 대비), BABILong, 메모리 레벨 ablation, M3 optimizer(ViT)

지금까지 'layer 하나' 만 바뀌 왔다 — 그것을 훈련시키는 '배경' 을 설계 대상으로

문제 1 — layer 하나만 바뀌 왔다

Titans/Miras/Atlas/TNT 는 전부 'sequence layer 하나' 의 성분 (objective · retention · optimizer · 훈련경제학) 을 바꾼 것 .

그런데 그 layer 를 바깥에서 훈련시키는

AdamW · momentum · backpropagation 은 한 번도 설계 공간에 안 들어온 ' 고정 배경 ' 이었다 .

문제 2 — LLM 은 정적이다 (anterograde amnesia)

배포 후 지식이 사는 곳은 딱 둘 : 지금의 context window(KV cache) 와 pre-training 때 얼어붙은 MLP weights.

context 정보가 장기저장소 (FFN) 까지 못 가고 window 가 밀려나면 사라진다 = 새 장기기억을 못 만드는 기억상실 .

핵심 통찰 — backprop 도 memory 다 (이 슬라이드만 가져가도 됨)

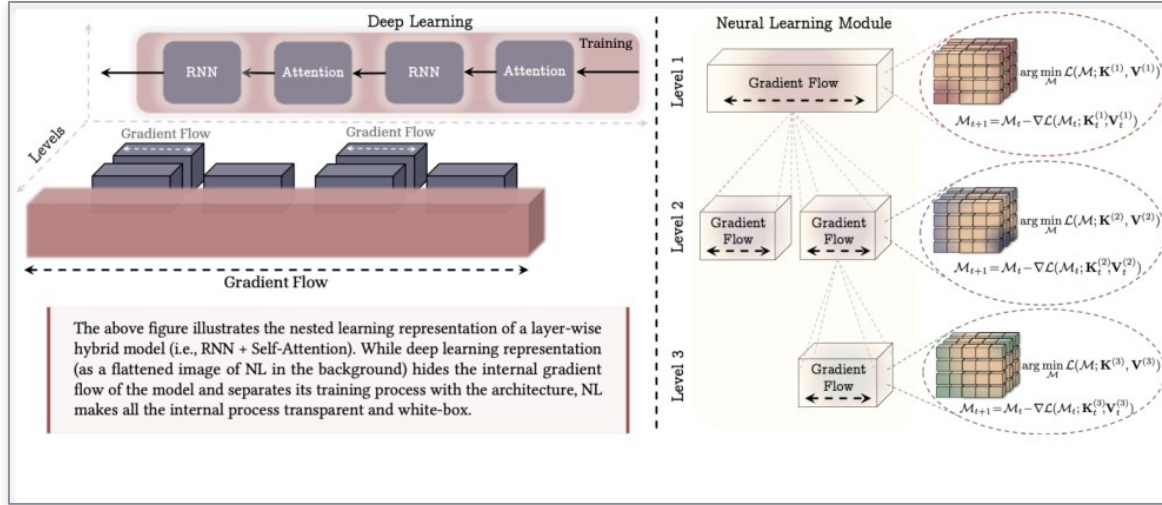
' 모델과 그것을 학습시키는 절차는 서로 다른 두 물건이 아니라 , 하나의 중첩된 다층 최적화 시스템이다 . '

그 안에서 backpropagation · momentum · Adam · Muon 이 전부 gradient 에 대한 associative memory 다 . 특히 backprop 은 자기 target 을 스스로 만드는 self-referential memory 라 병렬화가 안 된다 .

그래서 질문의 단위가 바뀐다

'layer' 가 아니라 ' 모델 전체 + 훈련 절차 전체 ' 가 하나의 설계 대상 . 각 level 은 자기 context flow 를 자기 주기로 압축하는 associative memory. (TNT 가 남긴 ' 서로 다른 주기의 계층 ' 힌트를 조직 원리로 승격 .)

학습 자체를 '중첩된 최적화 (= 메모리) 의 층' 으로 본다



Nested Learning: 모델과 그 학습절차를 중첩된 최적화 문제들의 집합으로

출처 : 2512.24695 Fig.2

핵심 재해석

optimizer 가 하는 일 (과거 gradient 를 모아 다음 스텝을 정함) = '연상 메모리' 다 . 그럼 momentum 은 선형 메모리 , Adam · Muon 도 메모리의 특수형 .

optimizer $\equiv$ associative memory over gradients

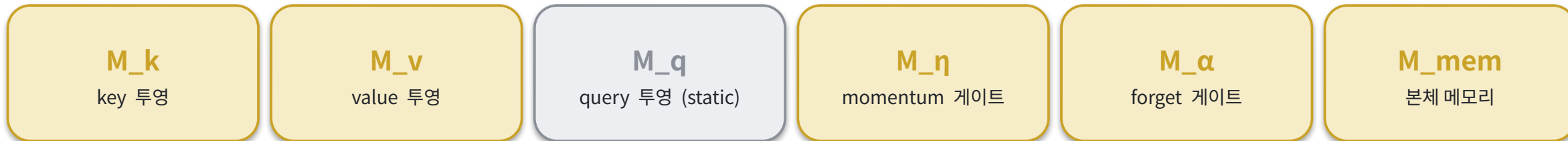
그래서 무엇이 열리나

optimizer 를 '더 표현력 있는 메모리' (Deep GD · delta-momentum · M3) 로 바꿀 수 있고 , 이 최적화 층을 여러 개 쌓으면 higher-order in-context learning 이 된다 .

ICL 에 대한 주장

in-context learning 은 큰 모델에서 '창발' 하는 마법이 아니라 , 최소 2 개 레벨의 중첩 최적화를 가지면 구조적으로 (structural) 따라 나오는 성질이다 — Fig.7 의 레벨 ablation 이 이를 뒷받침 .

투영 · 게이트까지 메모리 — 자기 갱신 규칙을 스스로 수정



↑ 6 개 메모리 중 q 만 static(고정), 나머지 5 개는 추론 중 갱신됨

무엇이 새로운가

Titans 는 본체 메모리 M_mem 만 갱신했다 . HOPE 는 ' 무엇을 key/value 로 삼을지 (투영) ' 와 ' 얼마나 기억 · 잊을지 (게이트) ' 까지 메모리로 만들어 test-time 에 함께 갱신한다 .

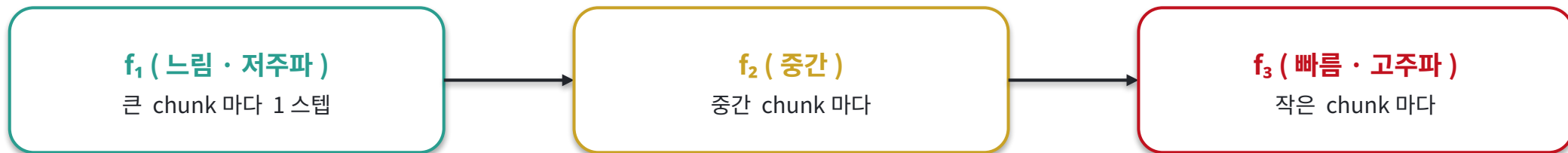
어떻게 갱신하나

target 을 스스로 만든다 (self-target: $\hat{v}_\square = M_\square(v_t)$). 갱신은 DGD-with-weight-decay, gradient 는 full backprop. 출력은 갱신 전 (chunk 시작) 메모리로 읽는다 .

의미

Titans 의 ' 고정된 투영 ' 을 풀어 , 모델이 자신의 학습 규칙 (무엇을 어떻게 기억할지) 을 데이터에 맞춰 바꾸게 만든 것 . q 를 고정으로 남긴 건 안정적 읽기 기준을 유지하기 위함 (추론).

long/short 를 ' 연속 주파수 ' 의 메모리 블록들로



forward: $y = MLP^{f_3}(\dots MLP^{f_1}(x))$ · 각 블록은 자기 주파수 경계에서 gradient 를 모아 1 스텝 갱신

TNT와의 관계

TNT의 global/local(2 단계)을 임의 개수의 주파수로 일반화. 느린 블록 = 장기 문맥, 빠른 블록 = 국소 세부. 블록 간 전달은 '가중치 복사'가 아니라 forward 활성화로.

meta-learned 초기상태

각 블록의 '리셋 지점'(초기 상태)은 사전학습으로 meta-learn. 느린 블록이 빠른 블록의 리셋 초기값을 제공. 이건 런타임 diff 전달이 아니라 시작점 설정 (→ B6 Sleep 가 그 diff 전달을 맡음).

HOPE = self-modifying Titans + CMS

자기수정 (어떻게 기억할지) + 연속 주파수 (언제 기억할지)를 합치면, '모든 부품이 어떤 주파수의 메모리'인 하나의 중첩 시스템 = HOPE가 된다.

HOPE 의 렌즈로 앞 4 편을 한 문장씩 다시 읽기

Titans

원래 : 메모리 = gradient 로 학습되는 신경망 (surprise+momentum+forget).

HOPE: HOPE: 그 optimizer(momentum 등) 마저 메모리 → 아키텍처와 optimizer 가 같은 것의 두 레벨 .

Miras

원래 : 4 축 (attentional bias · retention · architecture · algorithm) 으로 설계공간을 정의 .

HOPE: HOPE: 'algorithm' 축 (momentum/Adam/Muon) 이 사실 메모리 → architecture · algorithm 축을 하나로 통합 .

Atlas

원래 : feature map 으로 용량 확장 + Muon(2 차) optimizer 로 메모리 학습 .

HOPE: HOPE: Muon 류 = gradient 의 ' 표현력 큰 메모리 ' → M3(더 표현력 있는 optimizer) 로 일반화 .

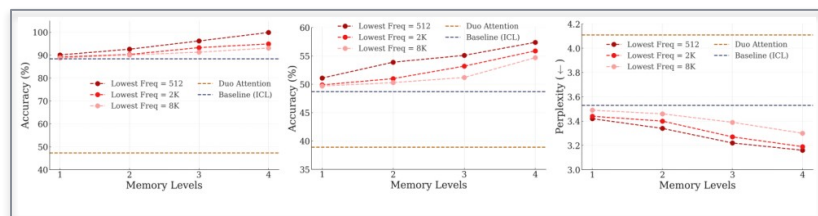
TNT

원래 : global/local(장기 / 단기) 2 단계 메모리 + train/serve chunk 정렬 .

HOPE: HOPE: long/short 를 CMS 의 ' 연속 주파수 ' 스펙트럼으로 확장 — 2 단계가 아니라 연속체 .

한 문장 : HOPE 는 ' 모든 부품 (투영 · 게이트 · optimizer · 장단기 메모리) 이 어떤 주파수의 메모리 ' 라는 하나의 중첩 시스템으로 앞 4 편을 통합한다 .

레벨이 ICL 을 , 표현력 큰 optimizer 가 학습을 개선한다



메모리 레벨 ↑ → ICL(NIAH) 성능 ↑ (structural)

출처 : 2512.24695 Fig.7

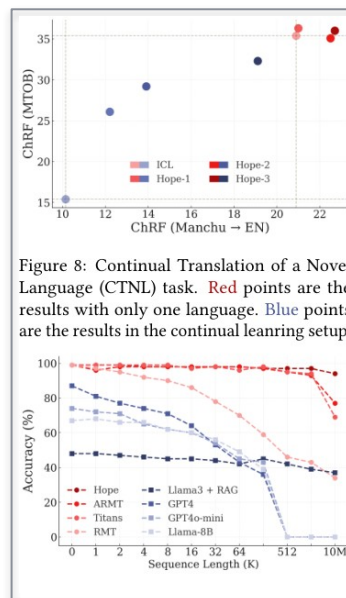
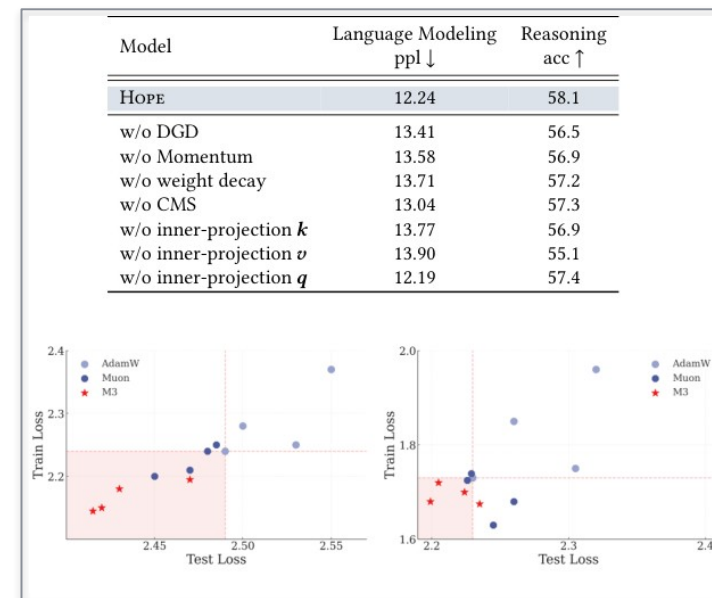


Figure 8: Continual Translation of a Novel Language (CTNL) task. Red points are the results with only one language. Blue points are the results in the continual learning setup.

BABILong: HOPE 가 대형 모델과 견줄

출처 : 2512.24695 Fig.9



M3 optimizer: ViT 에서 AdamW·Muon 대비 개선

출처 : 2512.24695 Fig.11

읽는 법

왼쪽 = 중첩 레벨을 늘릴수록 in-context 성능이 오른다 (ICL=structural 주장의 증거). 가운데 = 초장문 추론에서 경쟁력. 오른쪽 =optimizer 를 메모리로 보는 관점이 실제 학습 (ViT) 에서도 이득.

이 라인의 문제는 풀리기는커녕 커졌다 — 남은 절반을 Sleep 이 채운다

논문이 스스로 그은 한계 4 가지

- ① catastrophic forgetting 이해결 — forgetting 은 압축의 필연 (용량 유한). CMS 는 ' 어디서 · 얼마나 빨리 잊을지 ' 를 주파수 축에 재배치했을 뿐 , 원리적으로 풀 게 아니다 .
- ② level 설계가 경험적 — level 몇 개 · 주파수 얼마 · 무엇을 어디 놓을지 이론 없음 . query projection 을 memory 화하면 오히려 나빠지는 반례도 (잘못 놓인 level 은 해).
- ③ 이론이 재해석까지만 — optimizer 역공학은 존재 논증이지 유일성 증명 아님 , level 표현력 정리 없음 .
- ④ 서빙 경제학 · 스케일 미검증 — from-scratch 1.3B/100B, retrofit 도 Llama3-8B 까지 . decode wall-clock 전무 .

systems — batching 이 깨진다 (Part C 예고)

self-modifying Titans state = per-request mutable weights. KV cache(read-only, weight 공유 batch) 와 달리 shared-weight batching 이 깨진다 .

배치는 가능하나 각 요청의 arithmetic intensity 가 batch 크기 B 와 무관 → 대역폭이 배치를 조기에 닫는다 (never-average).

다음으로의 연결 — Sleep

신경생리학의 consolidation 은 2 단계 . NL 은 그중 online consolidation(입력 흐르는 동안 CMS 갱신) 만 구현했다 .

수면 replay 로 기억을 재조직하는 offline consolidation 은 명시적으로 범위 밖 . 남은 질문 : 빠른 level 지식이 예정된 갱신으로 덮이기 전에 어디로 옮길 것인가 ?

한 줄

HOPE = expressive optimizer(Delta Momentum · DMGD · M3) + CMS(주파수 스펙트럼 memory) + self-modifying Titans 를 직렬 결합 . ' 더 많은 layer 가 아니라 더 많은 level' 을 실증 — 단 짧은 recall gap 과 서빙 경제학은 미해소 , 그리고 offline consolidation 을 Sleep 에 넘긴다 .

Sleep — 자는 동안 지식을 ' 위로 ' 옮겨 파라미터를 키운다

핵심 발명 / 주장

1

wake/sleep lifecycle: 추론 (wake) 과 오프라인 통합 (sleep) 을 분리

sleep 은 별도 오프라인 단계 — 이때는 서빙하지 않고 그동안 배운 것을 정리 · 통합한다

2

Memory Consolidation = Knowledge Seeding: 배운 지식을 상위로 distill 해 low-rank expert 를 ' 성장 '

NL 의 online consolidation 은 초기상태 (reset) 만 meta-learn — ' 학습된 diff 의 위쪽 전달 ' 은 못 했다 . Sleep 가 그것을 함

3

Dreaming: 스스로 만든 데이터로 자기개선 (SEAL 식 RL)

teacher 가 ' 꿈 '(자기 생성 시퀀스) 을 만들고 RL 로 강화 — 외부 데이터 없이 능력 향상

논거 흐름

NL online consolidation(초기상태 meta-learn + circle-back) 은 고정 용량 한계 → 오프라인으로 파라미터 성장 → GKD(on-policy)+LTI(RL) → 새 low-rank expert 만 학습 (나머지 freeze) → synaptic pruning → Dreaming 자기개선

증명 (뒤에서 논문 그림 · 표로)

class-incremental(CLINC/Banking/DBpedia) 지속학습 , 메모리 레벨 효과 , BABILong 등으로 통합 효과 입증

NL 은 online consolidation 절반만 했다 — 잊히기 전에 옮길 곳이 없다

NL 이 남긴 숙제 4 가지

- ① catastrophic forgetting 미해결 — 여러 주기로 덮어쓰기를 지연해도, 모든 층의 갱신 시점이 겹치면 결국 덮어쓴다.
- ② NL 저자 스스로 'offline consolidation(자면서 정리) 은 범위 밖 ' 이라 명시 — 온라인 절반만 .
- ③ capacity 고정 — 유한 파라미터에 계속 압축해 넣으면 언젠가 옛 지식을 덮어쓴다 .
- ④ 입력을 끊고 자기 내부를 정리할 시간이 설계에 없다 .

anterograde amnesia (기억상실증)

배포된 LLM 의 지식은 두 곳에만 산다 :

- 세션 끝나면 사라지는 context window
- pre-training 때 얼어붙은 weight

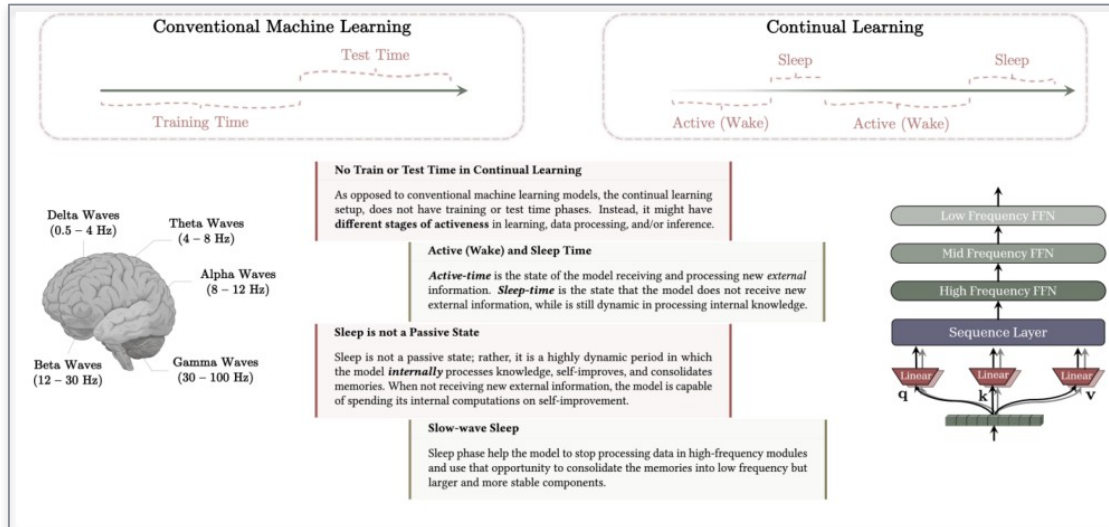
이 둘을 잇는 다리가 없다 = 새 장기기억을 못 만드는 ' 영원한 현재 ' !

남는 질문 : 빠른 level 의 지식이 예정된 갱신으로 덮어써지기 전에 , 그걸 어디로 옮길 것인가 ?

Sleep 의 답 (미리보기)

wake/sleep lifecycle 을 세워 , 잠자는 동안 그 지식을 더 느린 블록에 ' 새 expert 로 위쪽 distillation ' 하고 , forgetting 을 regularization 이 아니라 ' capacity 성장 ' 으로 재정의한다 . 그 발화 시점이 정확히 CMS 의 chunk 경계 .

축을 하나 더 — 수식의 축이 아니라 ' 시간 ' 의 축



lifecycle + inference 번역

sleep = 수동적 멈춤이 아니라 능동적 내부 처리 (외부 입력 끊고 , 고주파 모듈 기억을 느리고 안정적인 성분으로 굳힘).

= 서빙 fleet 에 붙는 주기적 백그라운드 job. 낮 = 요청 처리하며 fast memory 에 write, 밤 = 트래픽 끊고 compaction · GC. 단 대상이 로그 /cache 가 아니라 파라미터 .

online-only 의 3 결함

- ① 추상화 수준 그대로 (추가 압축 없이 옮겨 capacity 또 씀)
- ② 선택적 · retrieval 의존 (자주 부르는 것만 강화)
- ③ context 에 갇힘 (상위 통합 안 됨).

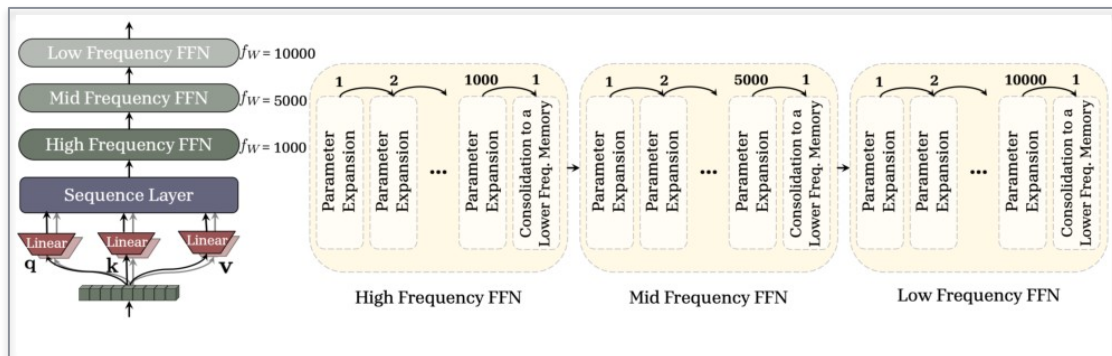
Figure 1 — conventional(train/test 분리) vs continual(Active↔Sleep 주기 교대)

출처 : 2606.03979 Fig.1

세 가지 핵심 주장

- ① continual learner 엔 train time 도 test time 도 없다 ('test time' 이란 단어의 마지막 잔재를 지움).
- ② catastrophic forgetting 은 근본적으로 capacity 문제 → ' 덮어쓰지 말고 키워라 ' .
- ③ sleep 은 2 단계 : Memory Consolidation(NREM) + Dreaming(REM).

sleep 은 학습되지 않고 chunk 경계에 고정된다



구조 (NL 의 CMS 를 그대로 수입)

sequence-mixing layer(attention/Titans) + 여러 주기의 MLP 블록 사슬 .

각 블록은 토큰마다 error 기여분을 누적하고 , 자기 chunk 경계에서만 그 누적을 파라미터에 한 번 적용 .

sleep 발화 시점

어떤 블록이 자기 갱신 경계에 도달하면 , 갱신하기 직전에 그 블록의 지식을 '한 단계 느린 블록'으로 먼저 옮긴다 .

Figure 7 — High/Mid/Low FFN 이 1k/5k/10k 토큰 주기로 배열된 CMS 사슬

출처 : 2606.03979 Fig.7

왜 여기가 catastrophic forgetting 이 터질 자리인가

주기가 중첩 → 다대일 (빠른 블록 10 번 갱신 동안 느린 블록 1 번). 같은 크기의 느린 메모리에 10 번을 반복해 써 넣게 됨 → 바로 그 지점이 CF 가 터질 자리 → 다음의 parameter expansion 이 필요 .

덮어쓰지 말고 키워라 + 지식을 '위로' 증류 (upward)

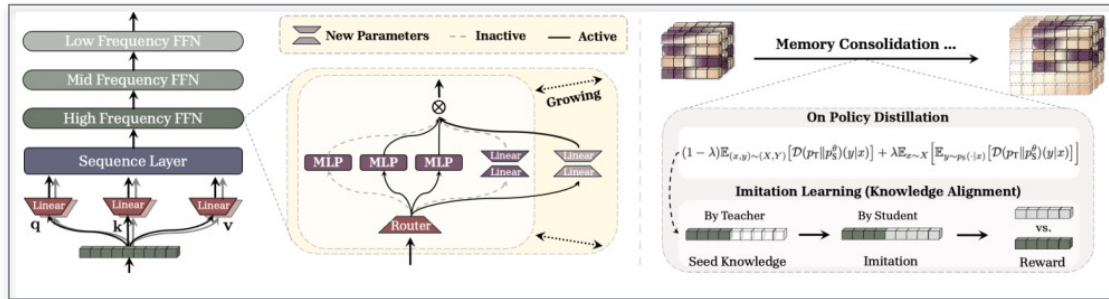


Figure 2 — 파라미터를 늘리고 (capacity), 사라질 지식을 새 공간에 증류
출처 : 2606.03979 Fig.2

① parameter expansion

consolidation 받는 블록에 새 파라미터를 연다 (새 MLP/Linear expert 하나). ' 자기 전에 파라미터를 키운다 '.

② Knowledge Seeding

지식 추상을 고주파→저주파 메모리로 이전 . GKD distillation(teacher 데이터 + student rollout) + Imitation Learning(RL).

반전 — upward distillation

보통 distillation 은 큰 teacher→ 작은 student 로 내려보내는데 , 여기서 방향이 반대 (작은 것 → 큰 것). 그래서 upward distillation.

B5(HOPE) 와의 연결 — 빠진 조각을 채운다

HOPE 의 CMS 는 ' 느린 블록이 빠른 블록의 리셋 초기값 ' 을 줄 뿐 (런타임 diff 전달 아님). Sleep 는 오프라인에서 빠른 메모리가 배운 것을 새 expert 로 만들어 느린 블록에 ' 실제로 ' 없다는 = 위쪽 전달 .

새 low-rank expert 에만 쓰고 · self-generated 로 배우고 · 옮긴 뒤에만 지운다

1a. parameter expansion — '덧어쓸 수 없게' 만든다

각 MLP = 여러 expert 를 가진 sparse MoE. 빠른 블록 지식을 옮길 때 새 low-rank expert 하나 추가 ($A d \times r, B r \times d$). 옮겨지는 지식은 오직 이 새 파라미터에만 → 옛 지식 물리 자리를 안 건드림 (정규화로 '잊지마' 부탁이 아니라 애초에 덧어쓸 수 없게).

[구현] 미래 expert 전부를 초기화 시점에 사전할당하고 mask 만 씌운다 → 'expert 추가'는 실제로 mask 하나 여는 것. shape 가 정적으로 유지 (컴파일 그래프 · kernel · checkpoint 안 흔들림).

1b. Knowledge Seeding — teacher/student 가 누구인가

teacher = expansion 전 모델 (빠른 블록이 아직 안 덮인, 그 지식을 든 자기 자신).

student = 확장 + 빠른 블록 갱신 후 (빠른 블록 지식은 잃고, 느린 블록에 빈 expert 하나 받은 상태).

distillation 이 student 출력을 teacher 로 당김 — 움직일 수 있는 건 새 expert 뿐. GKD = teacher 데이터 + student on-policy rollout. student 전부 freeze, 새 expert 만 학습.

1c. LTI + synaptic-pruning reset

distillation 만으로 '아는 것 ≠ 쓰는 것' — teacher 를 약하게만 흉내. LTI(RL): teacher 데이터 앞부분 주고 이어쓰게 해, 잘 쓰면 보상.

보상 = semantic(frozen reward model, 의미 같으면 1) + absolute(편집거리 토큰 유사도).

끝나면 reset: 빠른 블록의 과거 expert 들을 지워 capacity 회수 = 'eviction 전 write-back' cache 정책.

순 효과 — 하나의 불변식

빠른 메모리는 작고 유연하게, 느린 메모리는 단조 성장하며 '신선한 expert 로만' 쓰인다.

자기 생성 데이터로 자신을 고쳐 쓴다 (REM) — 단 knob 은 meta-learn 안 됨

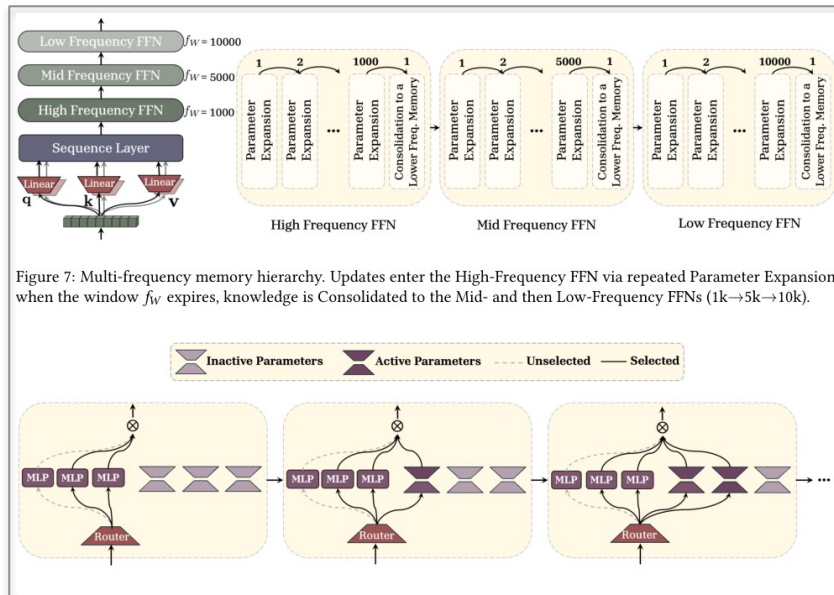


Figure 8 — sleep 사이클마다 router 가 expert 를 골라 갱신 , random expert 로 novelty 주입

출처 : 2606.03979 Fig.8

Dreaming (SEAL 위에 시연 , plug-in)

모델이 성능을 올려 줄 합성 데이터 (dream) 를 스스로 생성하는 self-modification. 5 단계 :

생성 (모든 MoE router 가 random expert 추가로 컴 =novelty 주입) → 선별 (gradient importance) → 적용 (격리 복사본 LoRA fine-tune) → 보상 (실제 task 개선되면 1) → 강화 (ReST^EM, value network 없는 EM RL).

consolidation 먼저 · **dreaming** 나중 : 취약 지식을 새 expert 에 격리한 뒤 반복 self-training.

세 번째 regime + 정직한 이음새

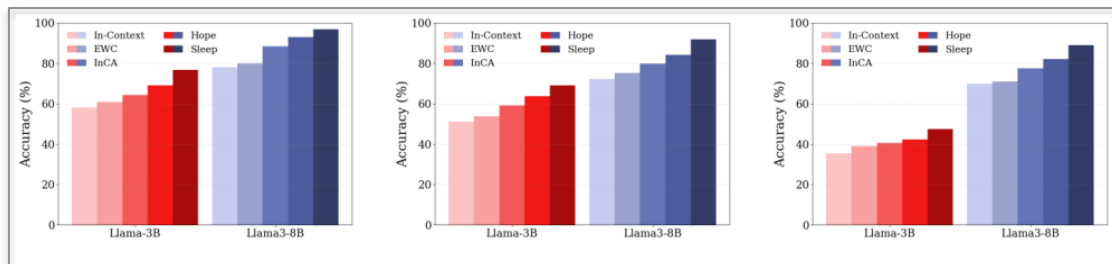
regime 1 pre-training · 2 wake(CMS 갱신) · 3 sleep(배포 중 주기적 ' 진짜 gradient 훈련 ' , 신세계).

caveat: sleep 기제 (혼합계수 · reward model · ReST^EM) 는 pre-training 에서 end-to-end meta-learn 안 됨 — 이 라인 최초로 핵심이 미분되는 inner loop 가 아니라 알고리즘 wrapper. 실험 대부분이 Llama/Qwen 에 graft.

왜 이 순서 (consolidate→dream) 인가

갓 배운 취약한 지식을 새 expert 에 격리해 둔 다음 dreaming 의 반복 self-training 을 돌려야 , 그 지식이 덜 잊힌다 .

방향은 일관 · 서빙 형태가 바뀐다 · 단 실증은 1.3B 에 멈춤



결과 (방향 · 비율)

sleep 단계 늘릴수록 단조 개선 . SQuAD SEAL 프로토콜에서 Sleep>SEAL(Dreaming 제거하면 폭락). few-shot ARC: Llama-1B ICL 0%→SEAL 72.5%→Sleep 80%. 수학 Qwen3-8B AIME-24 Sleep 79.2 > GRPO 76.4.

효율 : step 당 SFT 의 4 배지만 , 목표 성능 wall-clock 은 SFT 가 3.6~4.8 배 더 걸림 → 사는 건 싼 step 이 아니라 step·sample 효율 .

Systems 함의 (Part C 예고)

inference 가 forward-only 가 아님 (RMW 트래픽 =cost model 새 항). session state = KV cache 가 아니라 per-user weight-delta(=multi-tenant LoRA 서빙). sleep = 서빙에 붙는 스케줄 훈련 job, 매번 새 버전 .

Figure 3 — class-incremental(CLINC 등): consolidation 없은 Hope 가 ICL·EWC·순정 Hope 상회

출처: 2606.03979 Fig.3

6 편의 완성형 (개념적)

state 는 모든 시간규모에서 weight · 모든 블록은 같은 associative memory · inner optimizer 는 아키텍처와 대등한 설계면 · 훈련은 어디서나 chunk-anchored · lifecycle 은 wake/sleep. Titans 가 연 'test-time 학습' 이 여섯 편째에 train/test 경계 자체를 지운다 . (단 from-scratch 실증은 아직 1.3B.)

PART C

**System modeling (우리 기여) — 이 배포는 서빙에서
얼마나 드나**

무엇으로 뽑았고, 왜 신뢰할 수 있나 — HAT HW schema + HATIR

HAT HW schema — 가속기의 '디지털 트윈'

무엇 : 가속기를 기술하는 정식 스키마 (SoT). die/PE 내부 (SM→ALU→ISA), 메모리 계층 (HBM/L2/SRAM 의 대역폭 · 용량), peak FLOPS 를 담는다.

input: 벤더 spec (여러 번 교차검색해 신뢰). output: .hw JSON '트윈'.

검증 : schema 의 verify(committee). — 증거 : 수정 안 한 vLLM 이 이 트윈 (가짜 device) 을 진짜로 믿고 구동.

HATIR — 순수 비용 (cost) 모델

무엇 : HAT IR 기반 비용 모델. 트윈과 workload 를 받아 roofline 기반 시간 · 에너지를 낸다.

input: HW twin(.hw) + workload trace (연산 DAG). output: 시간 · 에너지의 ideal 하한.

특징 : magic derate · regression fitting 없이 독립 물리 요소로 분해 (냉정한 하한).

신뢰성 원칙 — 절대값이 아니라 'trend 정합'

정합 = 여러 시나리오에서 trend 일관성 (절대 수치 매칭 아님). ZigZag 등 기존 도구와 7/7 일치, 42 curves / 7 아키텍처 / 12 families 에서 97.6% trend 일치, fitting 없음.

→ 이 파트의 모든 수치는 'ideal roofline 하한' (달성 가능한 최선) 으로 읽어야 한다.

실험 조건 (전 Part C 공통) — 모두 명시

HW twin — H100

HBM3 대역폭 3.35 TB/s
peak 0.99 PF (989 TFLOPS, bf16)
ridge 295 FLOP/byte
on-chip SRAM 10 TB/s
baseline L2 50 MB

model — 1 HOPE block (B=1)

d = 2048
self-mod Titans 5 memory:
{M_k, M_v, M_η, M_α: d², M_{mem}: 2d²}
CMS 3 레벨 (각 d→4d→d MLP)
state(RMW) 134 MB · weights 277 MB

가정 (assumptions)

decode C=1 (토큰당)
kernel launch time = 0
gradient 는 chunk 시작 상태에서 평가
출력은 갱신 전 (pre-update) read
수치 = max(FLOP/peak, bytes/BW)

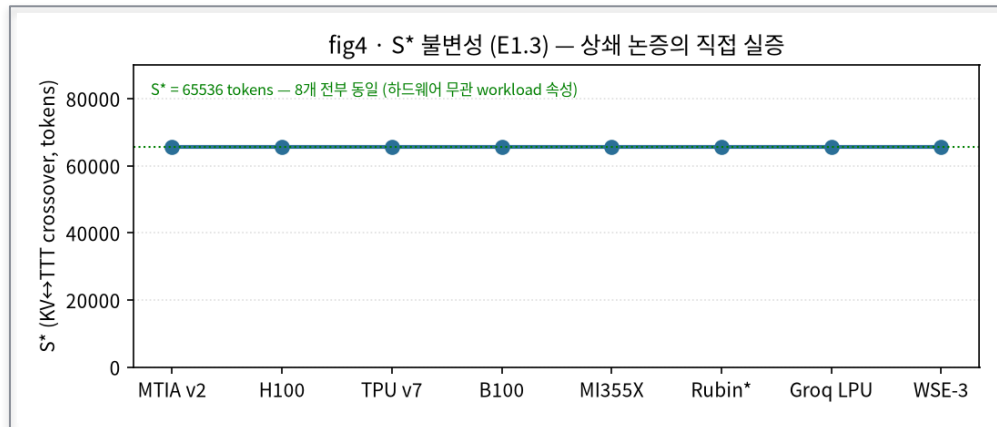
학습은 compute-bound, 디코드는 memory-bound

학습 (training) — compute-bound

chunk C 를 크게 잡아 병렬 → 연산 / 바이트 (arithmetic intensity) 가 높다.
roofline 의 오른쪽 (연산 한계) → 더 빠른 연산기가 이득.

디코드 (decode) — memory-bound

chunk=1, 토큰마다 '전체 상태 (state+weights)' 를 read-modify-write.
arithmetic intensity 가 낮다 → roofline 의 왼쪽 (대역폭 한계).



$S^*(KV↔TTT \text{ 교차점}) = 65536 \text{ tokens}$ — 8 개 가속기 전부 동일 = 하드웨어가 아니라 workload 의
성질

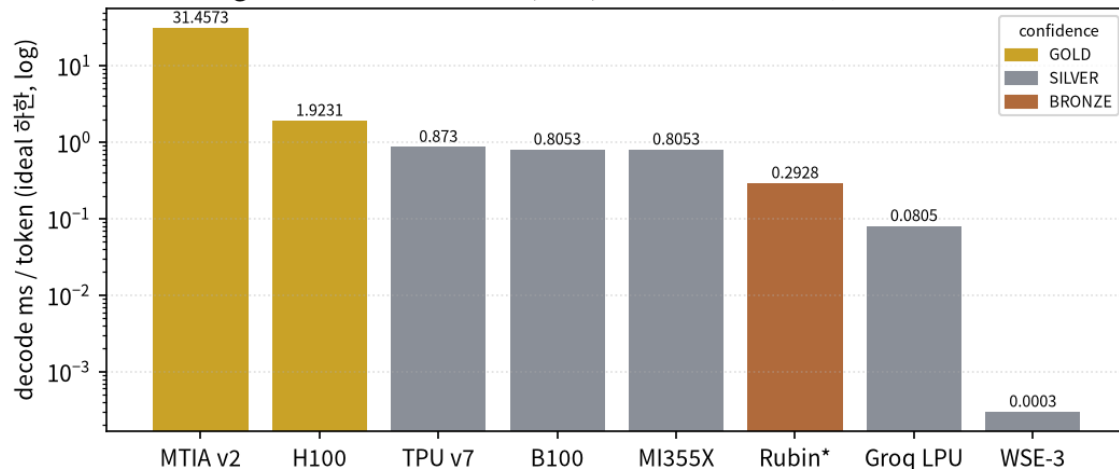
왜 중요한가

메모리 상태를 갱신하는 이 계열은 디코드에서 구조적으로 memory-bound 다. 그리고 그 경계 (S^*) 는 하드웨어를 바꿔도 변하지 않는다 (invariant). 즉 '어느 칩을 써도' 디코드 비용은 메모리 대역폭이 지배한다.

→ 이후 슬라이드 : 실제 8 칩 측정 · HOPE block roofline · 메모리 기술 (zHBM/PIM/SRAM) · 청사진.

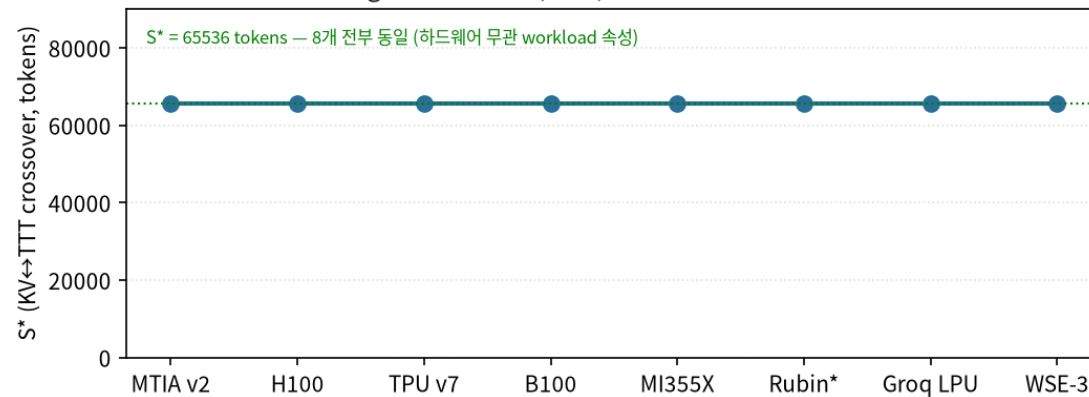
5 자릿수 스프레드, 그러나 대부분 memory-bound

fig1 · 토큰당 decode 비용 (E1.1) — 5자릿수 스프레드, state 배치가 축



토큰당 decode 시간 (ideal 하한, log) — MTIA v2 ~31ms 에서 WSE-3 ~0.0003ms 까지

fig4 · S* 불변성 (E1.3) — 상태 논증의 직접 실증

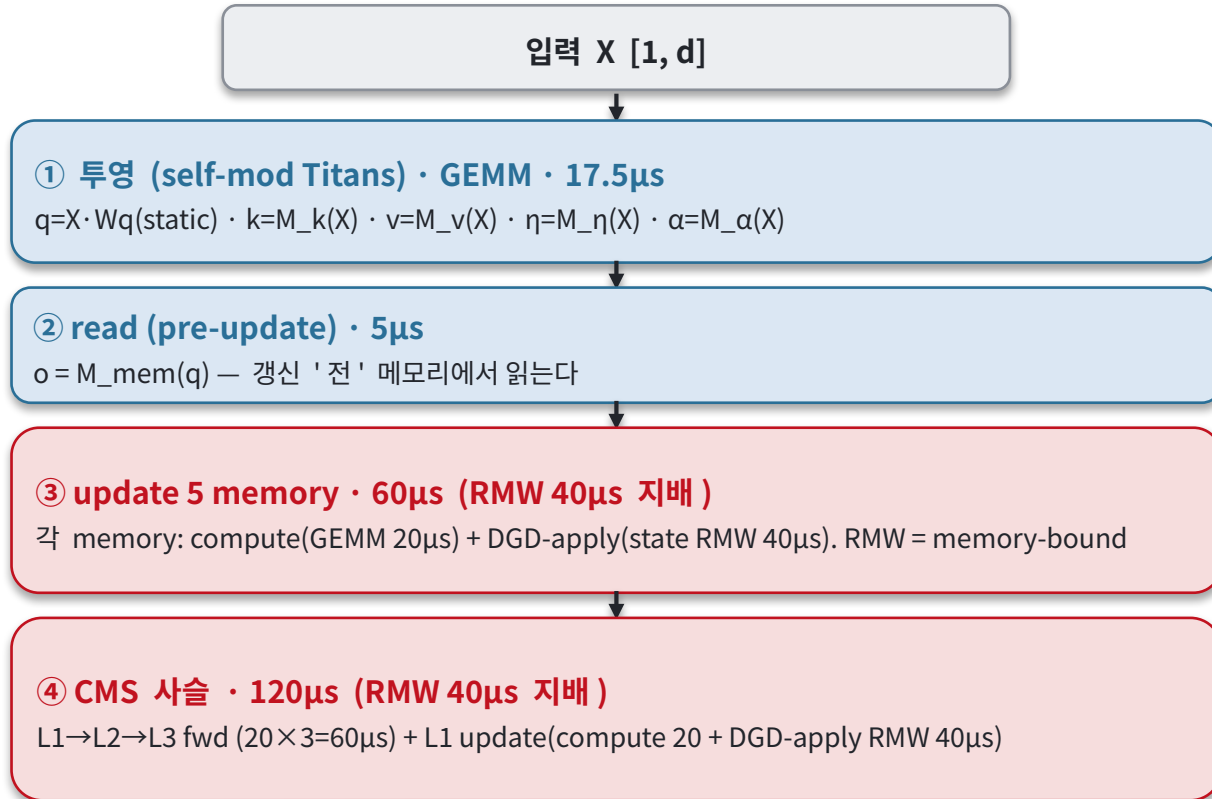


S* 불변 : 8 개 아키텍처가 같은 교차점 — workload 고유 성질

읽는 법

8 칩 중 7 개가 memory-bound(대역폭이 결정). 예외는 WSE-3(Cerebras) — 거대한 on-chip SRAM 으로 상태를 상주시켜 compute 쪽으로 넘어가는 'knee' 가 생긴다. 각 칩 spec 은 HAT hw twin 으로 검증 (die/PE 구조까지).

1 개 block 의 전 연산 (H100 트윈 , d=2048, C=1) — 21 개 op 을 4 단계로



op-time 분해 (합계 202.8μs, ideal 하한)

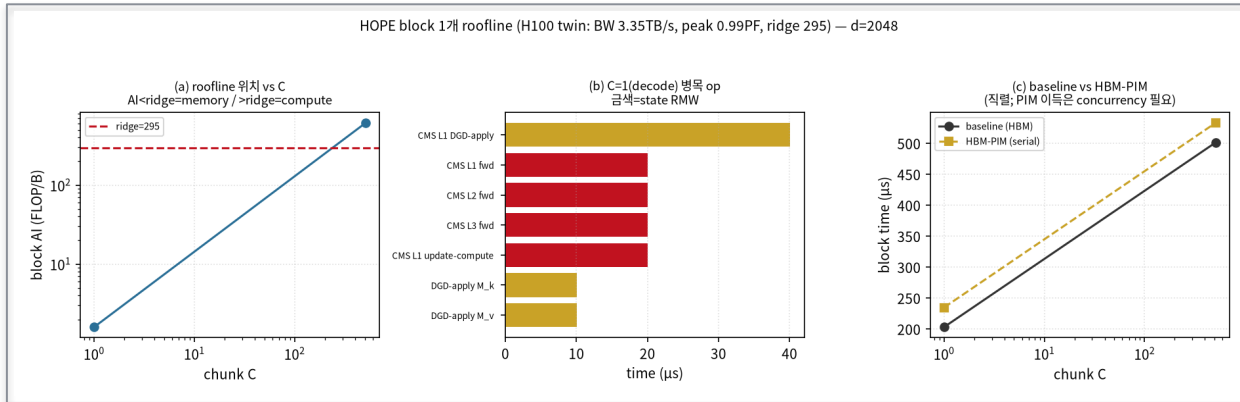
① 투영 GEMM ($q \cdot k \cdot v \cdot \eta \cdot \alpha$)	17.5μs
② read $o=M_{\text{mem}}(q)$ [pre-update]	5.0μs
③ update-compute (5 memory, GEMM)	20.0μs
③ DGD-apply (5 memory, state RMW)	40.0μs
④ CMS L1/L2/L3 fwd ($d \rightarrow 4d \rightarrow d$)	60.0μs
④ CMS L1 update-compute (GEMM)	20.0μs
④ CMS L1 DGD-apply (state RMW)	40.0μs

memory-bound RMW 합 = 80μs (전체의 40%)

정리

compute(GEMM) 는 텐서코어로 싸지만 , state RMW(빨강) 80μs 가 단일 최대 비용 덩어리 — 이게 decode 를 memory-bound 로 만든다 .

decode C=1 은 roofline 왼쪽 (memory-bound) — state 를 매 토큰 HBM 에서 RMW



(a) block AI vs chunk C — ridge 295 아래 = memory-bound (b) op 별 시간 (c) baseline vs HBM-PIM

H100 트윈 spec

HBM3 3.35 TB/s · peak 0.99 PF · ridge 295 FLOP/B · SRAM 10 TB/s · L2 50 MB

왜 memory-bound 인가

block 의 arithmetic intensity = FLOP/bytes = $1.08e9 / 6.80e8 \approx 1.6$ FLOP/byte.
H100 ridge = 295 FLOP/byte 보다 한참 낮다 → 대역폭이 병목 (roofline 왼쪽).

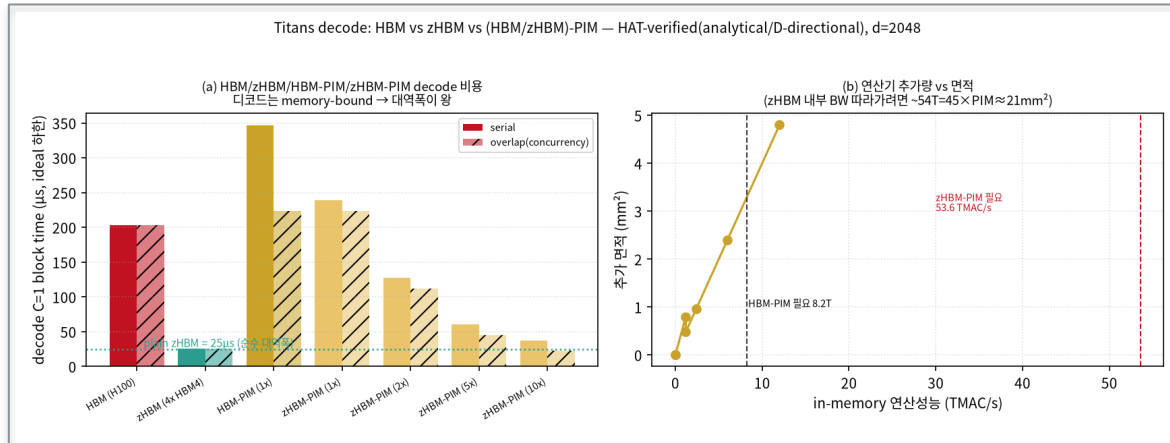
병목의 정체

state(134MB, RMW 되는 부분) + weights(277MB) 를 매 토큰 HBM(3.35TB/s) 에서 읽어야 한다. 이상적으로도 $411MB / 3.35TB/s \approx 123\mu s$ 는 대역폭에 묶인다 → block 당 $\sim 203\mu s$.

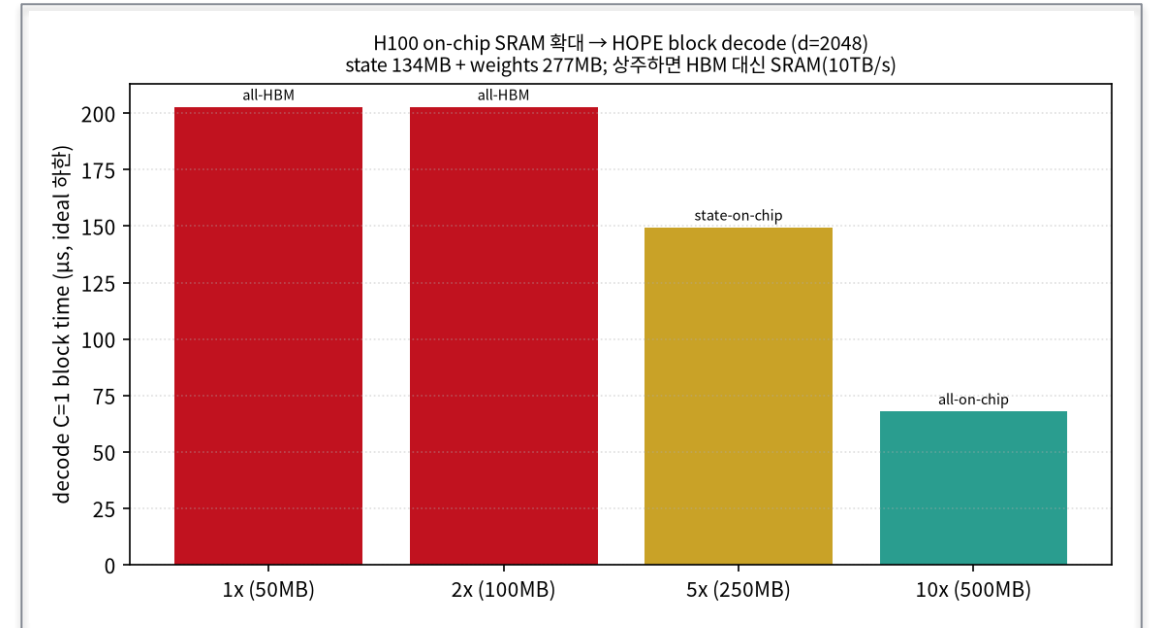
그래서 다음 (C4)

compute 를 키워도 소용없다. 답은 '대역폭 ↑' (zHBM) 또는 'state 를 on-chip 에 상주' (SRAM). PIM 은 in-bank 연산이 이 RMW 엔 너무 느리다.

memory-bound 라면 답은 '대역폭' 과 '상주'



HBM/zHBM/HBM-PIM/zHBM-PIM decode 비용 + zHBM 내부 연산 추가 면적

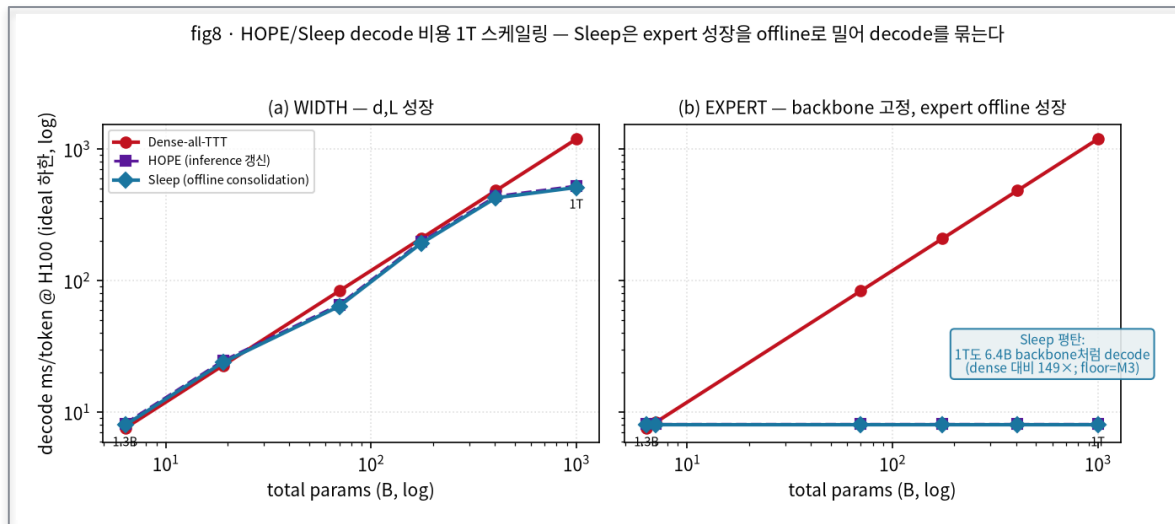


on-chip SRAM 1x→10x: state 가 상주하는 순간 (5x~) HBM 대신 SRAM

결론

zHBM(대역폭 ↑) 이 HBM-PIM(in-bank 연산이 너무 느림) 보다 유리. SRAM 은 '문턱' 구조 — state($\approx 134\text{MB}$) 가 on-chip 에 들어가는 5x(250MB) 부터 이득 (203→150 μs), 10x(500MB) 면 weights 까지 전부 상주.

논문의 방향 × 모델링 결과 = 어디로 가야 하나



HOPE(메모리 확장)+Sleep(파라미터 성장) 스케일링 — 상태 · 파라미터가 함께 자란다

한 줄

이 계열을 키울수록 서빙은 ' 메모리 문제 ' 가 된다 — 대역폭 · 상주 · 소프트웨어가 스케일의 열쇠.

① 논문 방향이 memory 압력을 키운다

HOPE(self-mod+CMS) 로 state ↑, Sleep(expert 성장) 로 파라미터 ↑ → 디코드 memory-bound 가 구조적으로 심해진다.

② 대역폭 우선

zHBM 류 고대역폭 메모리가 1 순위 지렛대 (PIM 보다 유리).

③ state 상주 설계

on-chip 용량을 state 가 들어갈 문턱 이상으로 — Cerebras 의 knee 가 그 증거.

④ 소프트웨어 효율이 프론티어

cross-vendor 소프트웨어 효율 (HATIR) 이 남은 최대 변수.

6 편을 한 흐름으로 , 그리고 서빙 비용까지

Titans	test-time 에 학습되는 신경망 메모리 (surprise+momentum+forget) 를 연다 .
Miras	그 설계를 4 축 공간으로 조직화 — 무엇을 · 무엇을 잊고 · 어떤 구조로 · 어떤 알고리즘으로 .
Atlas	문맥 단위 기억 (Omega) 과 용량 확장 (feature map)+Muon 으로 표현력을 키운다 .
TNT	train/serve chunk 정렬과 global/local 계층으로 ' 어떻게 배포하나 ' 를 답한다 .
HOPE	optimizer=memory·self-mod·CMS 로 앞 4 편을 하나의 중첩 시스템으로 통합 .
Sleep	wake/sleep 생애주기로 지식을 오프라인에서 위로 옮겨 파라미터를 키운다 .
우리 기여	이 배포는 디코드에서 memory-bound — 8 칩 측정 +HOPE roofline+zHBM/SRAM 로 비용과 청사진을 냈다 .